CAN THO UNIVERSITY

COLLEGE OF INFORMATION AND COMMUNICATION TECHNOLOGY

Project – Fundamental Topics

A MOBILE PERSONAL FINANCE MANAGEMENT APPLICATION SUPPORTED BY A LARGE LANGUAGE MODEL — PERFIN

Major:	Software Engineering
Cohort:	49
Course class:	CT239H M01
Advisor:	Dr. Phan Phuong Lan
Student:	Nguyen Thanh Trong
Student ID:	B2305615

Semester 3, academic year 2025–2026

Can Tho, 2026

ABSTRACT

PERFIN is a personal finance management prototype that aims to reduce data-entry effort while preserving correctness and verifiability. The system ingests transactions from text, voice and receipt images; it normalizes input, extracts records and requests confirmation before writing to PostgreSQL. SQL and deterministic algorithms compute balances, budgets, trends, anomalies, cash-flow and goals, while the LLM only interprets intent, fills parameters and narrates already-computed facts. The main contribution is an architecture that separates language generation from the financial core, together with an evaluation protocol for correctness and numeric faithfulness. Three reproducible experiments on a labeled dataset of 5,265 transactions show that the local parser reaches macro-F1 0.177 over the full set; on a stratified 63-sentence sample the LLM (Gemini) reaches 0.607 against the parser's 0.204; and correction retrieval raises holdout accuracy from 0% to 68.19%. These are category-classification measurements, distinct from the 31/31 quality gate, which is a pass/fail test on well-structured sentences. OCR/STT accuracy, numeric faithfulness and production readiness are not yet demonstrated and remain to be measured on a version-locked dataset.

Keywords: personal finance management, large language model, data analysis, PostgreSQL, Redis, verifiable systems.

Contents

List of Tables

List of Figures

LIST OF ABBREVIATIONS

Abbreviation	Meaning
AI	Artificial Intelligence
API	Application Programming Interface
CER	Character Error Rate
CRUD	Create, Read, Update, Delete
CSV	Comma-Separated Values
DB	Database
ERD	Entity–Relationship Diagram
F1	Harmonic mean of precision and recall
FR	Functional Requirement
HTML	HyperText Markup Language
HTTP	HyperText Transfer Protocol
IEEE	Institute of Electrical and Electronics Engineers
IQR	Interquartile Range
JSON	JavaScript Object Notation
KV	Key–Value
LLM	Large Language Model
NFR	Non-Functional Requirement
OCR	Optical Character Recognition
PDF	Portable Document Format
PERFIN	Personal Finance — the name of the prototype presented in this report
REST	Representational State Transfer
SQL	Structured Query Language
STT	Speech-to-Text
TC	Test Case — a test-case identifier at the specification level
TTL	Time To Live
UAT	User Acceptance Testing
UI	User Interface
VND	Vietnamese Dong
WER	Word Error Rate

STATUS AND RESULT CONVENTIONS

This report clearly distinguishes implemented components, measured results and behavior that exists only at the design level. This convention prevents expectations or smoke tests from being presented as completed quality evidence.

Table 1: Convention for levels of information confidence in the report

Label	Meaning	Usage
Implemented	A corresponding component exists in source code, migrations or tests.	Proves existence; does not automatically prove quality.
Measured	A command, timestamp and reproducible run result exist.	May be used to report figures in the results section.
Target	A desired threshold used as an acceptance criterion.	Must not be presented as an achieved result.
Design target	Expected behavior after in-scope fixes are completed.	Must be tested before being converted to “measured”.
Out of scope	Not a goal of the current project.	Mentioned only under limitations or future work.

CHAPTER 1: INTRODUCTION

1.1. PROBLEM STATEMENT

1.1.1. Background

Personal finance management requires users to record transactions regularly, classify them correctly, and understand the meaning behind their income–expense history. Financial literacy is directly linked to the ability to plan and make long-term decisions [1]. However, traditional recording tools typically demand multiple manual steps: selecting the transaction type, entering the amount, choosing a category, a wallet, and a time. When data entry becomes burdensome, records are prone to being missing or misclassified, which in turn makes every downstream report less reliable.

Natural-language input such as "ăn phở sáng nay 45k bằng Momo" (had phở this morning, 45k, paid via Momo) reduces the number of manual steps, but introduces new technical challenges. The system must understand Vietnamese currency notation, relative dates, multiple transactions within a single sentence, approximate wallet names, and categorization context. Receipt images and voice input add further OCR/STT noise on top of this. If inferred results are written directly into the database, a small error can corrupt balances, budgets, and every insight derived afterward.

1.1.2. The Analysis and Interpretation Problem

Charts and filters answer well-defined questions such as "how much was spent this month" effectively. The research value of PERFIN does not lie in wrapping SQL queries with a chatbot. The system needs to detect phenomena that are hard to observe from a single number: gradually rising trends, anomalous spending days, the rate at which cash flow is being depleted, recurring fees, or goals that have drifted off track.

These phenomena must be computed by testable algorithms. An LLM can understand questions and narrate results, but it is not the source of truth for financial computation. Delegating the entire analysis to an LLM makes results hard to reproduce and increases the risk of numbers appearing that are not grounded in the underlying data.

1.1.3. Research Questions

This project focuses on answering three questions:

1. How can text, image, and voice input be converted into structured transactions while preserving data correctness?
2. Which deterministic algorithms are suitable for producing explainable insights

from personal financial history?

3. Where should the LLM be involved to increase naturalness and personalization without taking over computation or unilaterally modifying data?

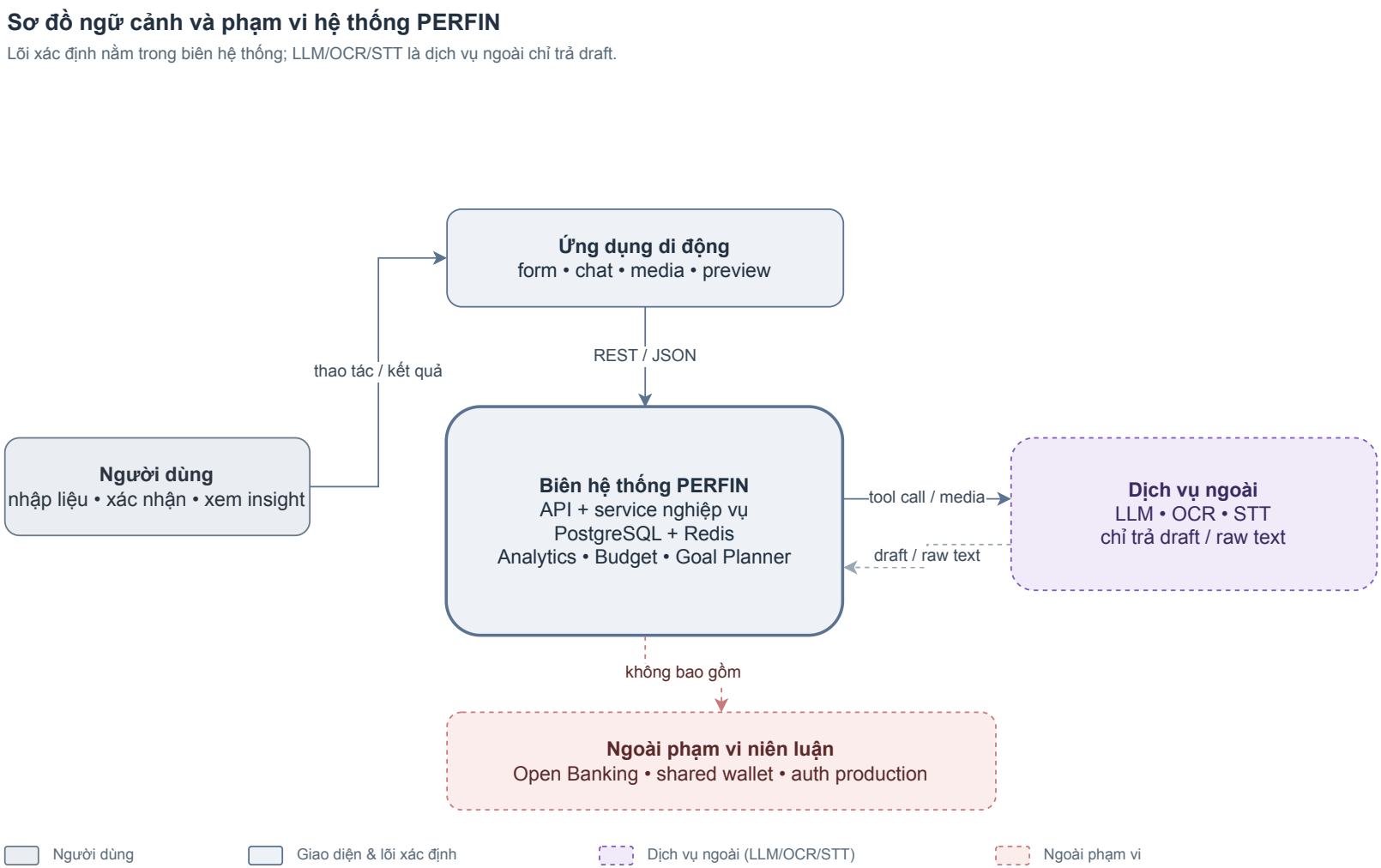

Figure 1: Context diagram and scope of the PERFIN system. The user interacts with the application; LLM, OCR, and STT providers are merely external supporting services.

Figure 1 positions PERFIN as a data and algorithm management system. Bank connectivity, shared wallets, production-grade authentication, and high-availability infrastructure are outside the scope of this basic-topic undergraduate project.

1.2. OBJECTIVES

The overall objective is to build and evaluate a personal finance management prototype in which structured data and deterministic algorithms form the foundation, while the LLM serves as a controlled layer for language understanding and narration.

Table 2: Project objectives and evaluation criteria

Code	Specific objective	Evidence and acceptance criteria
O1	Build a financial data model with keys, constraints, indexes, and atomic transactions.	Migrations, ERD, rollback tests; critical update flows must complete fully or leave data unchanged.
O2	Build a multimodal extraction pipeline with clarification and confirmation.	Labeled dataset, tool-call logs; no write to the database when a required field is missing.
O3	Implement trend, anomaly, runway, recurring, correlation, budget, and goal analytics.	Formula tests against normal data, edge-case data, and hand-computed results.
O4	Restrict the LLM to intent understanding, tool invocation, and narration of facts.	Tool schema, numeric-faithfulness checks and fallback; no figures added beyond the facts.
O5	Evaluate the operational viability of the prototype.	Integration tests, p50/p95, provider error rates, and UAT must have reproducible logs before being reported.

The thresholds in Table 2 are **acceptance targets**, not results that are automatically assumed to have been achieved. The report only labels a result as “measured” when the command, environment, and reproducible output are available.

1.3. PROJECT SCOPE

1.3.1. Scope of Implementation

Table 3: In-scope implementation and out-of-scope content

In scope	Out of scope
Income, expense, wallet-transfer transactions, and investment gain/loss at prototype level.	Open Banking integration and real bank synchronization.
Text, receipt-image, and voice input; preview and confirmation.	Training new foundation models, OCR, or STT engines.
Categories, correction feedback, suggestions, and confirmed re-tagging.	Shared wallets, group debt splitting, and real-time collaboration.
Budgets, recurring bills, insights, and financial goals.	Investment advice, credit scoring, or decisions made on the user's behalf.
A single demo profile `default_user` and a data upgrade path.	Registration, JWT, authorization, and production-grade multi-user isolation.
Redis, worker, and fallback mechanisms for demo/testing purposes.	High availability, centralized monitoring, and uptime commitments.

The focus of this project is the data model, the correctness of data transformations, and the analytical algorithms. The mobile interface serves to capture input and demonstrate usability; full CRUD coverage for every entity is not the primary measure of academic contribution.

1.3.2. Assumptions and Limitations

- The default currency unit is VND; monetary data is stored as a decimal type in PostgreSQL. Numbers generated by the LLM must not be used as an authoritative source.
- Statistical conclusions are only meaningful when the number of observations is sufficiently large. The system must lower its confidence level or decline to draw a conclusion when data is too sparse.
- A persona only changes the manner of expression, not the facts, computations, access rights, or confirmation conditions.
- The prototype does not replace a financial expert; its suggestions only support the organization and understanding of personal data.

1.4. CONTRIBUTIONS

This project aims to deliver five verifiable contributions:

1. a data model with constraints and atomic update flows;
2. a pipeline combining an LLM, local parsers, fuzzy matching, and a human in the loop;
3. a set of deterministic analytical algorithms, decoupled from the language-generation layer;
4. a grounded-narration mechanism, in which the LLM only narrates already-computed facts;
5. an evaluation protocol that separates algorithmic correctness, AI quality, performance, and data-entry experience.

1.5. REPORT STRUCTURE

Chapter 1 presents the problem, objectives, and scope. Chapter 2 builds the theoretical foundation on financial data, LLMs, OCR/STT, and analytical algorithms. Chapter 3 specifies requirements, architectural design, the data model, processing flows, and testing. Chapter 4 reviews the objectives achieved, states limitations, and proposes directions for future development.

CHAPTER 2: THEORETICAL FOUNDATION

2.1. FINANCIAL DATA AND PREPROCESSING

2.1.1. Transaction, Balance and Cash-Flow Model

A minimally valid transaction has an amount $A > 0$, a transaction type, a date, a description, a category and a wallet within the user's scope. PERFIN distinguishes `income`, `expense`, `transfer`, `investment_inflow`, `investment_outflow` and `investment_pnl` because each type has a different effect on balances and reports. For a wallet w, the balance change caused by an income or expense transaction is

$$\Delta B_w = \begin{cases} +A, & \text{if income,} \\ -A, & \text{if expense.} \end{cases} \tag{2.1}$$

Over a period P, total income I_P, total expense E_P and net cash flow N_P are computed from transactions that have not been soft-deleted:

$$I_P = \sum_{t \in P,\, type(t)=income} A_t, \qquad E_P = \sum_{t \in P,\, type(t)=expense} A_t, \qquad N_P = I_P - E_P. \tag{2.2}$$

Transferring A from a source wallet s to a destination wallet d produces $\Delta B_s = -A$ and $\Delta B_d = +A$, so that $\Delta(B_s + B_d) = 0$. This invariant ensures a wallet transfer is not counted as income or expense. The prototype allows a negative wallet balance to reflect cash or an overdrawn account; the business rule is a positive amount, two distinct wallets belonging to the same user, not a "sufficient balance" condition. The debit, credit and history record must lie within the same `BEGIN/COMMIT` transaction; if one step fails, `ROLLBACK` restores the entire state.

2.1.2. Data Integrity and Lifecycle

PERFIN applies four layers of protection:

1. **Domain constraints:** transaction amounts, limits and contribution amounts must be valid; dates must not exceed the allowed business domain.
2. **Referential integrity:** transactions reference users, categories and wallets by foreign keys.
3. **Business integrity:** no transfer between the same wallet; no transaction written with a missing required field; soft delete does not remove the ability to recover.
4. **Atomicity:** every multi-table change or balance change together with its history is committed in full or not at all.

Indexes are placed on frequently filtered columns such as user, date, category, wallet and status. Money is stored as `DECIMAL(15,2)`; when passed to JavaScript it must be converted in a controlled manner to avoid string concatenation or unintended rounding. The cache is invalidated after commit. A cache error after commit must not be returned as a business error, because the client may retry and cause a duplicate effect.

2.1.3. Time-Axis Normalization and Observation Windows

Analysis by day, week or month must preserve the correct calendar axis. Let x_j be the total expense of the j-th calendar period in a window W. If that period has no transactions then $x_j = 0$; the period must not be removed from the series. Zero-fill prevents two distortions: compressing the time gaps inflates the slope, and taking only days with spending inflates the burn rate.

The observation window must always accompany the result. PERFIN currently uses 14 days for runway, 30 days for anomaly, 90 days for recurring items, 6 months for trend and 12 weeks for correlation by default. These are the prototype's starting parameters, not thresholds proven optimal across all users.

2.1.4. Durable Data and Transient State

PostgreSQL stores auditable business data. Redis stores short-lived state: transactions awaiting confirmation, the field being clarified, the list of ambiguous choices, caches and rate-limit counters. TTL expires conversation state through the `EXPIRE` mechanism. The in-process memory fallback serves development only; its data is lost on restart and is not a production configuration.

2.2. INPUT NORMALIZATION AND CLASSIFICATION

2.2.1. Normalizing Amount, Date and Transaction Structure

Natural input is converted into a typed draft before validation. The suffixes `k`, "nghìn", "ngàn" multiply the value by 10^3; `tr`, "triệu" multiply by 10^6. Relative dates are resolved against the local date of the request. A sentence with several items must produce an array of drafts, in which each element has its own `amount`, `type`, `description`, `transaction_date`, `category` and `wallet`. The parser or LLM only creates the draft; the service then checks for a finite number, the date domain, the category, the wallet and ownership before creating the preview.

2.2.2. Text Similarity and Safe Category Selection

Strings are stripped of diacritics, lowercased, cleared of non-alphanumeric characters and collapsed on whitespace. For two normalized strings a, b, the edit score uses the

Levenshtein distance D_{lev} [13]:

$$s_{edit} = 1 - \frac{D_{lev}(a, b)}{\max(|a|, |b|)}. \tag{2.3}$$

Let T_a, T_b be two token sets; the Dice coefficient [14] is

$$s_{dice} = \frac{2|T_a \cap T_b|}{|T_a| + |T_b|}. \tag{2.4}$$

If the smaller token set has at least two elements and is fully contained in the larger set, the containment score $s_{contain}$ lies in the range $[0{,}82; 0{,}94]$ according to the length ratio. The overall score in the source code is

$$s = \max(s_{edit},\ 0{,}92 s_{dice},\ s_{contain}). \tag{2.5}$$

The selection order is exact category name, exact alias, then fuzzy. Long input uses the threshold $\tau = 0{,}82$, input of no more than four characters uses $\tau = 0{,}90$. The best candidate must also beat the second candidate by at least a margin $m = 0{,}08$. If $s_1 < \tau$ or $s_1 - s_2 < m$, the system chooses "Other" or requests clarification rather than assigning an ambiguous result on its own. This formula serves FR-04 and does not replace the user's confirmation.

2.2.3. Learning from Feedback

When the user corrects a category, the system stores the original result–correct result pair together with the context. Exact/fuzzy corrections are considered before aliases and the LLM on the next round; a conflicting correction lowers confidence or requests confirmation. This is retrieval combined with adaptive rules, **not model fine-tuning**. The effect of feedback must be evaluated by accuracy or macro-F1 before and after on the same replay set.

2.2.4. OCR, Speech-to-Text and Provider Adapters

OCR comprises image preprocessing, text-region detection, recognition and post-processing. STT converts audio into a transcript. In PERFIN, the provider returns only raw text or a transcript along with the provider name and error status; the result then continues through the draft–validation–preview pipeline as text.

A receipt image may produce one aggregate transaction or several line-item drafts, but the current version **does not yet have an algorithm to automatically reconcile** the line-item total with the invoice total, tax and discount. The user must check the preview; a two-image smoke test is not a measurement of OCR accuracy. The transcript must also be shown before confirmation so that a recognition error does not

silently become an amount. The adapter allows a local provider to be replaced by a cloud provider without modifying the transaction service. When a provider fails, the system returns an error or allows manual entry; it does not generate sample data.

2.3. ANALYSIS AND PLANNING ALGORITHMS

Table 4: Deterministic algorithms and the features that use them

Algorithm	Input and output	Feature that uses it
Linear regression	Monthly expense series with zero-fill; slope, R^2, forecast	FR-07: trend
Z-score and IQR	Daily or per-item expense series; flags, scores, method	FR-07: anomaly
Cashflow runway	Total balance and 14 calendar days; burn rate, day count, depletion date	FR-07/FR-11: cash-flow alert
Recurring items	Description, amount, date; clusters, cadence, monthly estimate	FR-07/FR-11: subscription scan
Pearson	Two weekly series with zero-fill; positive correlation	FR-07: category relationships
Budget recommender	Monthly expense, income, limits; recommendations and overrun forecast	FR-09
Goal planner	Goal, contribution or repayment, deadline, interest rate; time, shortfall, what-if	FR-10

2.3.1. Linear Regression and Trend Forecasting

Let y_i be the total expense of period i, $x_i = i$ with $i = 0, \ldots, n-1$, and $\bar{x}$ and $\bar{y}$ the corresponding means. The least-squares (OLS) line $\hat{y}_i = a + bx_i$ [15] has

$$b = \frac{\sum_{i=0}^{n-1}(x_i - \bar{x})(y_i - \bar{y})}{\sum_{i=0}^{n-1}(x_i - \bar{x})^2}, \qquad a = \bar{y} - b\bar{x}. \tag{2.6}$$

The goodness of fit and the next-period forecast are

$$R^2 = 1 - \frac{\sum_i (y_i - \hat{y}_i)^2}{\sum_i (y_i - \bar{y})^2}, \qquad \hat{y}_n = \max(0, a + bn). \tag{2.7}$$

When a constant series makes the denominator of R^2 equal to 0, the source code returns $R^2 = 0$ and does not report a trend. The service adds a rising trend to the facts only when there are at least three months, $b > 0$, $R^2 \geq 0{,}5$ and the average percentage change between pairs with a positive prior sample reaches at least 10%. Zero-fill must be performed before the regression. The result describes a short-term linear trend; it does not model seasonality or future events.

2.3.2. Anomaly Detection with Z-score and IQR

For $n \geq 4$ expense values x_i, the mean, sample standard deviation and z-score are

$$\mu = \frac{1}{n}\sum_{i=1}^{n} x_i, \qquad s = \sqrt{\frac{\sum_{i=1}^{n}(x_i-\mu)^2}{n-1}}, \qquad z_i = \frac{x_i-\mu}{s}. \tag{2.8}$$

Quantiles are linearly interpolated over the sorted sequence. Let $IQR = Q_3 - Q_1$ and the upper threshold $U = Q_3 + kIQR$. PERFIN flags the high-expense side when

$$z_i \geq 2{,}5 \qquad \text{or} \qquad x_i > Q_3 + 1{,}5IQR. \tag{2.9}$$

If $s = 0$ then $z_i = 0$; if $IQR = 0$ then the IQR branch raises no flag. The output comprises the value, the date or label, z_i, the ratio to the mean and the method `z`, `iqr` or `z+iqr`. The result only asks the user to review; it does not prove fraud and the thresholds need to be calibrated on representative data.

2.3.3. Estimating Cash-Flow Runway

Let $B = \sum_w B_w$ be the current total balance, $W = 14$ the number of observed calendar days and $e_j \geq 0$ the spending on day j; days with no spending have $e_j = 0$. The burn rate and the number of remaining days are

$$\bar{e}_W = \frac{1}{W}\sum_{j=1}^{W} e_j, \qquad d = \left\lfloor \frac{B}{\bar{e}_W} \right\rfloor. \tag{2.10}$$

If $B \leq 0$, `daysLeft=0`; if $\bar{e}_W = 0$, the system does not forecast a depletion date (`null`). Otherwise, the depletion date equals the current date plus d. When there is a payday, the system finds its next occurrence in the calendar and compares it with the depletion date. For example, $B = 2{,}8$ million VND and a 14-day total expense of $1{,}4$ million VND give $\bar{e} = 100{,}000$ VND/day, so $d = 28$ days. This is an extrapolation that keeps the recent spending level constant; it does not yet account for upcoming income or an abnormal expense.

2.3.4. Mining Recurring Expenses

Only `expense` transactions not exceeding 500,000 VND are considered in the current configuration. After the descriptions are normalized, transactions with the same key are grouped into a cluster of m amounts a_i and dates t_i. The average amount, relative deviation and average cadence are

$$\bar{a} = \frac{1}{m}\sum_{i=1}^{m} a_i, \qquad \delta_i = \frac{|a_i - \bar{a}|}{\bar{a}}, \qquad \bar{\Delta} = \frac{1}{m-1}\sum_{i=2}^{m} |t_i - t_{i-1}|. \tag{2.11}$$

A cluster is stable when every $\delta_i \leq 0{,}15$. A cluster is a recurring candidate when it is stable and satisfies one of two conditions: $20 \leq \bar{\Delta} \leq 40$ days, or at least three occurrences. With $m = 2$, the cadence must still lie within the window. The monthly estimate is currently $\bar{a}$ under the assumption of one occurrence per month; the total subscription is the sum of the cluster estimates. Exact grouping helps limit false positives but may miss variant descriptions. The result is only a "recurring signal"; it does not automatically create a recurring bill or cancel a service.

2.3.5. Cross-Category Spending Correlation

For two weekly expense series $X = (x_1, \ldots, x_n)$ and $Y = (y_1, \ldots, y_n)$ on the same weekly axis, a category's missing week is filled with 0. The Pearson correlation coefficient [16] is

$$r = \frac{\sum_i (x_i - \bar{x})(y_i - \bar{y})}{\sqrt{\sum_i (x_i - \bar{x})^2 \sum_i (y_i - \bar{y})^2}}. \tag{2.12}$$

The function returns 0 if $n < 3$ or one series has zero variance. The current version considers only categories with at least four weeks of observation and reports only pairs with **positive** correlation $r \geq 0{,}6$; negative correlation is not yet reported. The system selects the pair with the largest r and must interpret it as "co-varying in the observed data", not infer a causal relationship.

2.3.6. Budget Recommendation and Forecasting

Let $s_{c,j}$ be the expense of category c in month j over m calendar months; a month with no activity has $s_{c,j} = 0$. The average and the base level with a buffer $\beta = 5\%$ are

$$\bar{s}_c = \frac{1}{m} \sum_{j=1}^{m} s_{c,j}, \qquad b_c = (1 + \beta)\bar{s}_c. \tag{2.13}$$

With monthly income Y, the default framework allots at most $C_{need} = 0{,}50Y$ to needs, $C_{want} = 0{,}30Y$ to wants and sets a target of $0{,}20Y$ for savings. Let g be the spending group (needs or wants), C_g its allocation ceiling and $\sum_{k \in g} b_k$ the total base limit of the categories in that group. Three strategies produce a raw limit L_c:

$$L_c = \begin{cases} b_c, & \text{category average,} \\ C_g \dfrac{b_c}{\sum_{k \in g} b_k}, & \text{50/30/20,} \\ b_c \min\left(1, \dfrac{C_g}{\sum_{k \in g} b_k}\right), & \text{hybrid.} \end{cases} \tag{2.14}$$

The result is rounded to a step of 10,000 VND; a positive value smaller than one

step still becomes 10,000 VND. Confidence follows the number of months in which the category had activity: high when at least 6, medium when 3–5, low when below 3; every recommendation also carries a warning if the observed history is under three months.

To forecast a limit overrun within the month, with S the amount spent after d_e days and a month of D days:

$$q = \frac{S}{d_e}, \qquad \widehat{S}_D = qD, \qquad d_{exceed} = \left\lceil \frac{L - S}{q} \right\rceil . \tag{2.15}$$

The system attaches `likely_to_exceed` when $\widehat{S}_D > L$. The formula assumes a constant spending rate, so it is only an early warning; any recommendation or limit change always requires the user's confirmation.

2.3.7. Goal and Debt-Repayment Planning

For a savings or purchase goal, let $T > 0$ be the goal amount, $C \geq 0$ the current amount, $R = \max(0, T - C)$ the remaining shortfall and $M \geq 0$ the monthly contribution. If $M > 0$:

$$n = \left\lceil \frac{R}{M} \right\rceil . \tag{2.16}$$

If the deadline is $n_D > 0$ months away, the required contribution is $M_D = \lceil R/n_D \rceil$, the monthly shortfall is $G = \max(0, M_D - M)$ and the shortfall at the deadline is $H = \max(0, R - Mn_D)$. When $M = 0$, the completion time is undefined; when $R = 0$, the goal is complete immediately. What-if simply re-runs the same function with $M' = M + E$, where E is the freed-up spending, and does not modify the original plan.

For debt repayment, L is the current outstanding balance, i_a the annual interest rate as a percentage, $r = i_a/(100 \times 12)$ the monthly rate and n the number of months. The level payment that completes on time is

$$P = \begin{cases} L/n, & r = 0, \\ \dfrac{Lr}{1 - (1 + r)^{-n}}, & r > 0. \end{cases} \tag{2.17}$$

The simulation uses the recurrence $L_{k+1} = \max(0, L_k(1 + r) - P)$ and adds $L_k r$ to the total interest. If $P \leq Lr$ in the first month, the balance does not decrease and the system returns the warning `NEGATIVE_AMORTIZATION`. If the debt is not settled within the default simulation limit of 600 months, the plan is not feasible within the horizon. The formula lets FR-10 clearly separate the computation from the LLM's

interpretation.

2.4. EVALUATION METHODOLOGY

2.4.1. Extraction and Classification Accuracy

For a field to be extracted, TP is a correctly predicted value, FP is a wrongly predicted or spurious value and FN is a ground-truth value that was missed:

$$Precision = \frac{TP}{TP + FP}, \qquad Recall = \frac{TP}{TP + FN}, \qquad F1 = \frac{2\,Precision\,Recall}{Precision + Recall}. \tag{2.18}$$

Exact match is achieved only when all required fields of a transaction are correct together. Macro-F1 averages the F1 of each category so that high-sample classes do not mask low-sample ones. These metrics must be computed on an evaluation set independent of the development prompts or rules.

2.4.2. OCR and STT Error and Business Impact

Here S, D, I are the number of substitutions, deletions and insertions respectively; N is the number of characters or words in the ground truth. The subscript c denotes character-level counts (CER) and w denotes word-level counts (WER):

$$CER = \frac{S_c + D_c + I_c}{N_c}, \qquad WER = \frac{S_w + D_w + I_w}{N_w}. \tag{2.19}$$

CER/WER measure raw text; transaction-field accuracy is what tells whether a media error corrupted the amount, date, merchant or line item. That is why a provider smoke test must not be presented as accuracy.

2.4.3. Algorithm Correctness, Grounding and Performance

Deterministic algorithms are checked with hand-computed expected values, the absolute error $AE = |\hat{y} - y|$ and invariants such as the unchanged total of a wallet transfer. For narration, let N_{all} be the number of numeric expressions in the response and $N_{supported}$ the number that map to facts or an equivalent formatting operation:

$$Numeric\ Faithfulness = \frac{N_{supported}}{N_{all}}. \tag{2.20}$$

$$Unsupported\ Rate = 1 - Numeric\ Faithfulness. \tag{2.21}$$

This is a **target evaluation method**; a checker covering every number is not yet fully implemented. Latency uses percentiles: p50 is the median, p95 is the value that 95% of samples do not exceed; text/media, cache hit/miss must be separated and the provider, hardware and number of runs recorded.

2.5. THE CONTROLLED SUPPORTING ROLE OF THE LLM

2.5.1. Function Calling Instead of Delegating Business Authority

The transformer uses attention to model token relationships and is the foundation of modern LLMs [2]. PERFIN leverages the ability to understand varied phrasing, but does not use the model to compute the ledger or statistics. Tools are declared by schema using the provider's function-calling mechanism [4]; the LLM proposes the tool name and arguments, while the application validates, creates the preview, awaits confirmation and only then executes. Structured output only guarantees the JSON shape, not that the category is correct, that the two wallets are distinct or that the date is within the valid domain.

2.5.2. Responsibility Boundaries and Grounding

Table 5: Responsibility boundary between the deterministic core and the LLM

Task	**Responsible component**	**Rationale**
Total income–expense, balance, limits	SQL and business service	Accurate and repeatable
Trend, anomaly, runway, correlation	Deterministic algorithm functions	Have formulas and independent tests
Budget and goals	Budget/Goal Planner	Have constraints and simulation
Understanding dates, currency units, multiple intents	LLM or local parser	A language/context problem
Selecting the tool and filling parameters	LLM, then the service checks	Bridge from utterance to API
Writing the interpretation/persona	LLM or template fallback	Must not change the facts
Write, delete, transfer, re-tag	Service after confirmation	Has data impact and needs audit

Insight facts must include the value, unit, observation window, sample count, method and a missing-data warning. The persona may change the tone in the spirit of behavioral design [12] but does not change the facts. The system currently separates the facts from the narrator and has a template fallback; a comprehensive numeric checker is still a target design that needs evaluation. The LLM does not connect directly to PostgreSQL and does not execute tools on its own.

2.5.3. Conditions for the Use of the LLM to Be Meaningful

The LLM is only valuable if it delivers a measurable improvement over the form and the local parser: higher field F1 or exact match, fewer clarification rounds or

actions, while latency, cost and the unsupported-number rate stay within acceptable thresholds. An ablation must use the same data and the same task. If it does not create a benefit sufficient to offset the risk, the direct flow or the local parser is preferred. This approach keeps the thesis focused on data and algorithms, with the LLM as a controlled communication layer.

2.6. INFRASTRUCTURE, TECHNOLOGY AND SELECTION RATIONALE

Table 6: Technologies, selection criteria and alternatives

Technology	Why it fits the problem	Boundary or alternative
React Native + Expo	One codebase for text, image, audio input and preview on mobile	Contains no formulas; separate native only needed when a device-specific requirement arises
Node.js + Express	I/O-oriented API; shares JavaScript with the pure functions and tests; suits a modular monolith	No need for the operational cost of microservices
PostgreSQL [7]	Foreign keys, `DECIMAL`, transactions and aggregate queries suit a ledger	Files or NoSQL struggle to guarantee the same level of constraints and rollback
Redis [8]	TTL, cache-aside, pending state and queue	Not a source of truth; the memory fallback is used only in development
BullMQ [9]	Scheduling, retry and job IDs for idempotent proactive tasks	A simple cron lacks the same queue/retry mechanism
Gemini or local parser	The LLM extends language coverage; the parser guarantees a testable fallback	The provider sits behind an adapter and does not lock the business logic
Local or cloud OCR/STT	Allows comparison of accuracy, latency, cost and privacy	Raw output always passes through the same validation/preview

Redis/BullMQ serve recurring reminders, runway scans, subscription scans, month-end reports and export cleanup. Jobs must be small and idempotent; a unique event key in the internal message prevents a duplicate effect on retry. Scheduled auto-backup does not yet have a corresponding worker and must not be regarded as operational.

2.7. RELATED RESEARCH AND APPLICATIONS

Research on financial literacy points to the role of planning and long-term decision-making [1]. Linear regression [15], z-score, IQR [10] and correlation [16] are explainable techniques, suited to the thesis goal because their output can be checked independently. OCR [5], STT [6] and multimodal LLMs [3] extend the input channels but do

not on their own guarantee the correctness of a financial record.

Commercial financial-management systems typically fall into three groups. The first is form/dashboard applications such as Money Lover [17] or MISA MoneyKeeper [18]: they have well-structured data and rich visual reports, but entry still follows multi-step forms and offers little support for everyday Vietnamese natural language. The second is financial chatbots: natural to talk to, but the numbers they return are often hard to trace back to a source and prone to drift. The third is deep analytic models: strong on statistics but lacking a confirmation loop for the everyday user. PERFIN does not compete on commercial completeness. The chosen gap is to combine a relational ledger, deterministic algorithms, a human in the loop and a bounded language layer.

Table 7: Gaps and PERFIN's approach to addressing them

Gap	**Risk**	**Approach**
Natural entry but easy to record incorrectly	Dirty data spreads into reports	Typed draft, validation, preview, confirm
Media has raw text but the fields are not certainly correct	Wrong amount, date, category	Measure CER/WER and field accuracy; confirm before writing
Chatbot returns numbers of unclear origin	Hallucination and hard to audit	Query tools, JSON facts, target grounding checker
Dashboard has numbers but lacks interpretation	Hard to notice patterns	Deterministic algorithms; the narrator only reads facts
Non-adaptive classification	Repeats the same error	Correction retrieval, fuzzy threshold and margin
Retry creates duplicate notifications	Poor experience and wrong data	Job ID, unique event key, idempotent handler

CHAPTER 3: APPLICATION RESULTS

3.1. SOFTWARE REQUIREMENTS SPECIFICATION

3.1.1. Overall Description

PERFIN is a mobile application prototype for entering, normalizing, managing and analyzing personal financial data. The prototype focuses on protecting data quality and verifying the algorithms; the chat interface is only a supplementary input gateway alongside the direct screens. All financial computation, constraint checking and data changes are performed by the backend. The LLM, OCR and STT do not access PostgreSQL directly and are not treated as authoritative sources of figures.

The current version operates with a single `default_user` profile. Per-user foreign keys create an upgrade path for later, but do not yet demonstrate that the system has authentication, authorization or multi-user isolation at production level. PostgreSQL is the durable source of business data; Redis stores state with TTL, cache and queue data, with process memory as a fallback in the development environment. The specification uses FR/NFR codes, acceptance conditions and test-case identifiers to provide traceability in the spirit of IEEE 830 [11].

Table 8: System actors

Actor	Role	Limitation
User	Enters data, edits/confirms/cancels the preview, manages budgets/goals and views reports.	Responsible for checking data inferred from language or media before saving.
Mobile application	Captures text, images, audio; calls the REST API; displays the preview, facts and results.	Does not access the database directly or hold the AI provider's secret keys.
API and business services	Validation, database transactions, computation and returning structured results.	Does not delegate computation or data-write authority to the LLM.
External AI services	Answer tool calls, OCR text or STT transcripts.	Results are untrusted input; they do not execute business logic themselves.
Background worker	Runs recurring reminders, runway/subscription scans, month-end insights and export cleanup.	Must be scoped per profile and prevent duplicate effects on retry.
Development administrator	Configures providers, migrations and demo data.	Not a product end-user actor.

The `TC-FRxx-yy` codes below are test-case identifiers at the specification level. A case is considered "measured" only when it has input data, an expected result, an environment and a corresponding log in Section 3.3.

3.1.2. Specific Requirements

3.1.2.1. Interfaces and external dependencies

- The mobile application communicates with the Express backend over REST/JSON; the application does not connect directly to PostgreSQL, Redis or the AI provider's API.
- Media input accepts the image/audio formats permitted by the backend, up to 10 MB per file. The transcript or raw text must be returned for the user to check.
- Gemini or the local parser handles intent understanding and field extraction; PaddleOCR/Google Vision and PhoWhisper/Google Speech are interchange-

able adapters. An unavailable provider must lead to an error or a real fallback, not a fabricated result.

- PostgreSQL holds business data and history that needs auditing. Redis holds pending/clarification state, cache and queues; losing Redis must not cause fabricated financial data.

3.1.2.2. Shared data and business rules

1. Transaction amounts for income/expense, wallet transfers, budgets and recurring bills must be finite positive numbers. Investment profit/loss values are finite non-zero numbers and may be signed.
2. Transaction, transfer and payment dates must not be in the future; a goal's target date must not be in the past when a plan is created.
3. Referenced categories and wallets must exist within the current profile's scope. The demo version does not yet have multi-user authorization.
4. Wallet balances **are allowed to be negative**. PERFIN records the ledger and warns about cash flow; it does not reject an expense/transfer merely because the source balance is insufficient.
5. Changes initiated by chat or media must pass through a preview and confirmation; direct actions from a form may be written after backend validation succeeds.
6. Operations affecting multiple records or balances must be within a single database transaction. A cache/hydration error after `COMMIT` must not be reported as a write error that makes the user retry.
7. Transactions use soft delete and can be restored; reports by default do not count deleted records.
8. Analysis results must include the data period, quantitative facts and the method. Missing data must return `null`/a warning, not infer strong conclusions.

3.1.2.3. Scope and priority

Table 9: Functional grouping by academic priority

Group	Functions	Role in the project
Data core	Transactions, wallets, categories, budgets, recurring bills and goals.	The main object of study.
Algorithm core	Parsing, matching, feedback, analytics, budget/goal planner.	The main contribution, to be described and tested.
LLM layer	Tool routing, extraction, clarification, narration and persona.	A bounded supporting component.
Demo support	Dashboard, management screens and export.	Demonstrates usability, not the focus.
Out of scope	Production auth, bank sync, shared wallet and high availability.	Future work, not a completion condition for the project.

3.1.3. Functional Requirements

Table 10: Functional requirements

Code	Requirement	High-level acceptance condition
FR-01	Enter transactions via natural-language text	Normalize one or more transactions, create a preview and do not write when required fields are missing.
FR-02	Enter via image and voice	Return a real raw OCR/transcript; confirm before the transaction pipeline; no claim of invoice-total reconciliation yet.
FR-03	Clarification and pending transactions	Store state for 5 minutes; allow supplement/edit/confirm/cancel; a preview can be claimed only once.
FR-04	Classification and learning from feedback	Prioritize safe exact/alias/fuzzy; record corrections after commit; do not learn when the result is ambiguous/conflicting.

Code	Requirement	High-level acceptance condition
FR-05	Manage financial data	CRUD with validation and soft delete; atomic balance update; allow negative wallet balances.
FR-06	Atomic wallet transfer	Debit, credit and history in the same transaction; two distinct wallets; no sufficient-balance check.
FR-07	Data analysis	Compute trend, anomaly, runway, recurring and positive correlation using deterministic algorithms.
FR-08	Grounded insight and persona	Return facts and interpretation; the numeric grounding checker is a design target, not a measured result yet.
FR-09	Budget and forecast	Compute progress/forecast; recommend based on history or a ratio framework; apply only on confirmation.
FR-10	Financial goals	Preview saving/purchase/debt plans and what-if; save only the exact previewed payload.
FR-11	Recurring bills and proactive tasks	Manage the schedule, atomic payment; worker retry with dedup and user scope.
FR-12	Data export	CSV works; the flow labeled PDF currently produces HTML; store history and clean up files past TTL.

3.1.3.1. FR-01 — Enter transactions via natural-language text

Purpose and trigger. Reduce manual entry while still enforcing the schema. The user sends a chat sentence or calls the analysis endpoint; the sentence may contain one or many transactions.

Input and processing. Non-empty text; the profile has wallets and categories. Target fields include the description, a positive amount, the `income/expense` type, a date not in the future, a category and a wallet. The orchestrator selects LLM structured output or the local parser, normalizes amount/date per Section 2.2.1 and maps the category/wallet. The backend applies the ledger rules of Section 2.1; the LLM only proposes arguments. A batch is written atomically only after the FR-03 preview.

Errors, output and postconditions. A missing amount or remaining ambiguity moves to clarification; a provider error uses a real fallback or returns an error. A

future date, an out-of-scope reference or a single wrong batch element rejects the whole. Before confirmation only normalized data, warnings and a `pending_id` are returned; only after confirmation is PostgreSQL changed, the pending consumed and the cache invalidated best-effort.

Acceptance criteria.

- `TC-FR01-01`: a sentence with all fields creates one preview and does not yet increase the transaction count in the DB.
- `TC-FR01-02`: a multi-transaction sentence extracts all of them and commits all-or-nothing.
- `TC-FR01-03`: a missing amount creates a clarification, not a pending that could be written wrong.
- `TC-FR01-04`: a future date or an out-of-scope reference does not change balances.

3.1.3.2. FR-02 — Enter transactions via image and voice

Purpose and trigger. Convert an invoice image or Vietnamese audio into checkable text and then reuse FR-01. The file must be of a backend-supported type and no larger than 10 MB.

Input and processing. A media adapter must be configured or a local provider must exist. The provider returns raw OCR/transcript together with the provider name; the system displays the result, extracts fields and then creates the choices/preview. The OCR/STT foundation is in Section 2.2.4; CER/WER and field accuracy are in Section 2.4.2. The current version may allow choosing to save the invoice total or individual line items, but **does not yet reconcile the item total against the invoice total**.

Errors, output and postconditions. A wrong-type/oversized file or a failed upload is rejected. A provider that returns no text must raise a real error, not create a mock. The API returns raw text/transcript, provider, context and extracted data; only after media confirmation and FR-03 is the transaction written.

Acceptance criteria.

- `TC-FR02-01`: a valid image returns raw OCR and provider, with no claim that line-item totals were matched.
- `TC-FR02-02`: the transcript is displayed, edited/confirmed and only then enters the parser.
- `TC-FR02-03`: a provider error creates no mock or pending.
- `TC-FR02-04`: a wrong-type file or one larger than 10 MB is blocked before the business logic.

3.1.3.3. FR-03 — Clarification and pending transactions

Purpose and trigger. Collect missing fields across multiple turns and prevent writing speculative data. FR-01/FR-02 create state when a field is missing, when there are multiple choices or when there is enough data to preview.

Input and processing. State is bound to the profile and includes the intent, pending fields, collected data, candidates and creation time; a pending holds one transaction/batch with metadata. State and pendings live in the KV store with a 5-minute TTL. Answers are merged and revalidated; editing the preview uses controlled get–update–set; confirmation uses an atomic `claim` so a `pending_id` is consumed only once.

Errors, output and postconditions. An expired pending, a wrong ID or an already-claimed one must not be written to the DB; a failed edit keeps the old preview when possible; canceling clears the state; a stale request does not overwrite a new preview. A valid confirmation clears the pending and hands the data to the transaction service.

Acceptance criteria.

- `TC-FR03-01`: an answer supplements the correct missing field and refreshes the TTL.
- `TC-FR03-02`: two simultaneous confirmations let only one request claim and write data.
- `TC-FR03-03`: a stale ID does not consume a new preview.
- `TC-FR03-04`: a failed edit or expired TTL does not change PostgreSQL.

3.1.3.4. FR-04 — Category classification and learning from feedback

Purpose and trigger. Choose an explainable category that adapts to personal corrections but does not learn from uncertain signals. The matcher receives the description, income/expense type, categories/aliases and corrections within the same profile.

Input and processing. Strings are stripped of diacritics, lowercased and whitespace-normalized. The order is exact name, exact alias, then fuzzy per Section 2.2.2; a threshold of 0.90 for very short input, 0.82 for the rest, and a minimum margin of 0.08. A correction close to the input may serve as few-shot context; the result still goes through validation/confirmation. Editing the category of a non-manual transaction records the old–correct pair best-effort after commit.

Errors, output and postconditions. Failing the threshold, two candidates too close, a conflicting alias or non-consensual corrections must fall back to “Other”/a request to choose. A failure to record feedback must not turn a committed transaction into a failure. The result includes category, confidence, match type and fallback

reason.

Acceptance criteria.

- `TC-FR04-01`: exact and exact-alias precede fuzzy.
- `TC-FR04-02`: a gap under 0.08 returns ambiguous/fallback.
- `TC-FR04-03`: short input under 0.90 is not auto-assigned.
- `TC-FR04-04`: a valid correction serves as an example; a conflicting correction does not force classification.

3.1.3.5. FR-05 — Manage transactions, wallets and categories

Purpose and trigger. Maintain a ledger that can be edited, soft-deleted, restored and queried without corrupting balances. The user performs CRUD on transactions, wallets and categories; filters support date period, type, category, wallet and pagination.

Input and processing. An income/expense transaction has a positive amount, a non-future date, and a valid category/wallet. Per Section 2.1.1, income adds to and expense subtracts from the balance. Create, edit of amount/type/wallet, soft delete and restore all apply the delta within the same DB transaction; changing the category does not change the balance; reports exclude records with `deleted_at`.

Errors, output and postconditions. Data outside the valid domain or references outside the profile are rejected; **the balance after an expense is allowed to be negative**. A cache error after commit is only logged. The API returns the hydrated transaction and the new balance; a restore reapplies the correct effect exactly once.

Acceptance criteria.

- `TC-FR05-01`: income/expense changes the balance by $+A/-A$ respectively.
- `TC-FR05-02`: editing amount/type/wallet applies the correct delta and rolls back entirely on error.
- `TC-FR05-03`: an expense larger than the balance is still valid and the wallet may go negative.
- `TC-FR05-04`: soft delete/restore reverses and applies the balance correctly exactly once.

3.1.3.6. FR-06 — Wallet transfers and special cash flows

Purpose and trigger. Record transfers and investment cash flows without leaving a dangling debit/credit. The amount must be positive and the date not in the future; a transfer needs two distinct wallets, and an investment flow needs the appropriate wallet type.

Input and processing. The service locks the wallets in order, subtracts A from the source, adds A to the destination and writes `wallet_transfers` within the same

`BEGIN/COMMIT`. An internal transfer preserves the zero-net-change invariant of Section 2.1.1. Non-zero investment profit/loss applies only to investment/savings wallets.

Errors, output and postconditions. Duplicate/missing wallets, a wrong amount/date or a wallet outside the profile cause a rollback. **There is no sufficient-balance check**; the source wallet is allowed to go negative. The system returns the transfer history and two balances that both reflect one transfer or both remain unchanged.

Acceptance criteria.

- `TC-FR06-01`: $B'_s = B_s - A$, $B'_d = B_d + A$ and the total balance is unchanged.
- `TC-FR06-02`: an insufficient source wallet is still allowed to go negative.
- `TC-FR06-03`: two identical/out-of-profile wallets are rejected before commit.
- `TC-FR06-04`: an error between debit, credit and insert rolls back all three effects.

3.1.3.7. FR-07 — Financial data analysis

Purpose and trigger. Turn transaction history into quantitative facts about trends, anomalies, cash-depletion risk, recurring spending and category relationships. The flow runs when the user opens a report/insight or a worker scans periodically.

Input and processing. Use only non-deleted transactions and zero-fill the time axis where needed. The Analytics Engine applies Sections 2.3.1–2.3.5: (i) trend reports an increase only with at least 3 months, a positive slope, at least a 10% average increase and $R^2 \geq 0{,}5$; (ii) anomaly needs at least 4 points, flagging the high-spending side when $z \geq 2{,}5$ or exceeding $Q_3 + 1{,}5IQR$; (iii) runway uses exactly 14 calendar days including zero-spend days, returns 0 days for a non-positive balance and forecasts no depletion date for a zero burn rate; (iv) recurring uses a 90-day window, a maximum amount of 500,000 VND, a 15% tolerance and a 20–40 day cadence; (v) correlation zero-fills 12 weeks, requires data in at least 4 weeks and reports only **positive** correlation $r \geq 0{,}6$, not $|r|$.

Errors, output and postconditions. Missing data returns `null`; edge branches handle a zero denominator/burn rate. A failed component is recorded in `degraded_components`; correlation must not be interpreted as causation. Facts JSON has a timestamp, values, period, method and warnings; the insight cache has a 10-minute TTL.

Acceptance criteria.

- `TC-FR07-01`: slope, forecast and R^2 match a hand-computed series.
- `TC-FR07-02`: z-score/IQR flags only the upper side and handles zero variance/IQR.

- `TC-FR07-03`: runway uses all 14 days including zero-spend days; a negative balance returns 0 days.
- `TC-FR07-04`: recurring obeys the count, amount and cadence; `TC-FR07-05`: only pairs with $r \geq 0{,}6$ are returned, negative pairs are not reported.

3.1.3.8. FR-08 — Grounded insight and persona

Purpose and trigger. Interpret FR-07 in understandable language without delegating computation to the LLM. The flow receives facts, the data period, `degraded_components` and the persona; the raw-facts endpoint must be usable independently of the narrator.

Input and processing. The backend sends structured facts to the narrator while also generating overall advice via deterministic rules. The persona only changes the phrasing; the response always includes facts, provider and metadata for cross-checking. The boundary is in Section 2.5.2, the grounding measurement in Section 2.4.3.

Errors, output and postconditions. If the LLM is unavailable, a template fallback is used or a clear error is returned, with no mock numbers generated. **A checker ensuring every number in the narration belongs to the facts is a design target**; numeric faithfulness is not yet claimed as achieved. The read flow does not write transactions, budgets or goals.

Acceptance criteria.

- `TC-FR08-01`: changing the persona does not change the facts of the same analysis.
- `TC-FR08-02`: a provider error uses a real fallback, keeps the facts and inserts no sample data.
- `TC-FR08-03` (design target): the checker detects numbers/units not supported by the facts.

3.1.3.9. FR-09 — Budget, recommendation and forecast

Purpose and trigger. Track limits, forecast overrun risk and recommend based on history rather than letting the LLM pick numbers. The user performs CRUD on budgets or requests `category_average`, `50-30-20`, `hybrid`.

Input and processing. The category must be an expense type, the limit positive, and the month/year valid. Progress computes spent, remaining and ratio; forecast uses spent/elapsed_days $\times$ days_in_month. The recommender per Section 2.3.6 averages over all months in the window including zero months, with a 5% buffer; needs/wants are capped at 50%/30% of income and saving at 20%. Hybrid only shrinks weights

when the total exceeds the cap. Under 3 months generates a warning; apply requires `confirmed=true` and an atomic upsert.

Errors, output and postconditions. Missing income makes ratio strategies fall back to the average. A wrong category/period/limit, an empty batch or one over 50 items, or a missing confirmation is rejected; one wrong item rolls back the whole batch. Only the confirmed apply endpoint changes `budgets`/`budget_history`.

Acceptance criteria.

- `TC-FR09-01`: a zero-activity month is still in the average denominator.
- `TC-FR09-02`: the forecast matches the formula and does not divide by 0.
- `TC-FR09-03`: under 3 months has a warning; missing income falls back correctly.
- `TC-FR09-04`: a missing confirmation is blocked; a wrong item rolls back the batch.

3.1.3.10. FR-10 — Financial goals and what-if simulation

Purpose and trigger. Compute contribution, completion time and scenarios for saving, purchase or debt goals using deterministic functions. Input includes goal type, positive target, non-negative current/contribution, a non-past date and an annual interest rate for debt.

Input and processing. The Goal Planner applies Section 2.3.7 to compute the shortfall, number of months, contribution to deadline, progress and warnings. For debt, the monthly rate equals the annual rate divided by 12; the annuity formula has a zero-rate branch and monthly simulation to compute interest/remaining balance; a payment not exceeding the first month's interest is flagged for negative amortization. What-if reruns the same function. The plan endpoint does not write to the DB and issues a payload-signed token valid for 15 minutes; creating/updating a plan field accepts only an unexpired token whose fingerprint matches.

Errors, output and postconditions. A wrong date/interest/amount, an expired token or an altered payload is rejected. A zero contribution has no deadline; a non-decreasing debt returns a warning. The preview returns the plan, progress, cashflow, what-if and token but writes no data; only a valid confirmation saves.

Acceptance criteria.

- `TC-FR10-01`: saving with/without a deadline matches the formula.
- `TC-FR10-02`: debt with zero/positive interest matches the simulation and detects negative amortization.
- `TC-FR10-03`: the preview creates no row; an expired token/different payload is blocked.

- TC-FR10-04: what-if does not change the saved goal.

3.1.3.11. FR-11 — Recurring bills and proactive tasks

Purpose and trigger. Manage recurring-spending schedules, record payments consistently and create non-duplicate reminders/insights on retry. A bill has a name, a positive amount, a weekly/monthly/quarterly/yearly frequency and a valid due day; a payment carries the just-read `period_due_date`; the worker receives a job name and user scope.

Input and processing. The due date is computed per cycle and clamped to end of month. Recurring suggestions use Section 2.3.4. A payment locks the bill, checks the period, creates an expense, subtracts from the wallet, records the payment and advances `next_due_date` within one transaction; deleting the plan still keeps the payments. The scheduler upserts the schedule, retries up to 3 times with exponential backoff; an event key/fingerprint and a unique index prevent duplicate message/export creation.

Errors, output and postconditions. A missing/stale period, a wrong schedule/amount/reference causes a rollback. An unsupported handler must raise a clear error; a retry keeps the same event key. The wallet after payment is allowed to go negative. The system returns the bill, payment, transaction, next period and worker statistics; a period has exactly one effect.

Acceptance criteria.

- TC-FR11-01: the four cycles produce the correct due date, including months missing days 29–31.
- TC-FR11-02: a payment updates four effects atomically and rolls back on error.
- TC-FR11-03: a stale-period request does not mistakenly pay a new period.
- TC-FR11-04: rerunning the same event key does not increase the message/file a second time.

3.1.3.12. FR-12 — Data export, file history and cleanup

Purpose and trigger. Let the user retrieve data by filter, track created files and clean up expired files. Only non-soft-deleted transactions belonging to the profile and matching the format/date-range/filter are exported.

Input and processing. UTF-8 CSV with BOM, escaping special characters, and storing history with a default 7-day TTL. The API flow named `pdf` currently produces `.html`, returning `text/html`; this is printable HTML, **not a binary PDF**. The worker deletes expired files and updates the history status. The `.pfbak` backup endpoint exists but has not been verified end-to-end and is therefore not evidence of production

recovery.

Errors, output and postconditions. If no matching data exists, do not create an empty file as if successful; a wrong format/filter, an expired file or an out-of-profile ID returns an error; do not return a fake URL when file writing fails. Cleanup makes a file undownloadable but keeps metadata per policy.

Acceptance criteria.

- `TC-FR12-01`: the CSV filters correctly, excludes soft-deletes and escapes special characters.
- `TC-FR12-02`: the PDF-labeled endpoint returns `.html`/`text/html` and must not be called a real PDF.
- `TC-FR12-03`: a file past TTL is cleaned up and the history reports it as unavailable.
- `TC-FR12-04`: only announce backup/restore after tests of integrity, checksum, rollback and table reconciliation.

3.1.4. Nonfunctional Requirements

The thresholds below are acceptance criteria or design targets, not necessarily current results. Actual measurements and missing measurements are separated in Section 3.3.2.

Table 11: Nonfunctional requirements and how to measure

Code	Requirement	Target	How to measure
NFR-01	Data correctness and atomicity	No partial write in create, update, delete, restore, transfer, recurring payment and batch apply; a valid negative balance is not counted as an error.	Step-by-step fault injection, before–after row/balance reconciliation and rollback checks.

Code	Requirement	Target	How to measure
NFR-02	Extraction accuracy	Lock separate thresholds for amount, type, date, category, wallet on an independent labeled set; do not infer from the 31 hard-coded cases.	Per-field precision, recall, F1, exact match and error analysis by sentence type.
NFR-03	Numeric faithfulness	**Design target:** the narration adds/changes no number or unit beyond the facts; not yet claimed as achieved.	A facts–response checker, numeric faithfulness and unsupported-number rate on a version-locked set.
NFR-04	Performance	Text p95 target ≤ 3 seconds; media p95 ≤ 8 seconds in the announced environment.	At least 30 runs/flow; record hardware, provider/model, cache hit/miss, p50/p95 and error rate.
NFR-05	Resilience	A provider error creates no fake data; a Redis error has a development fallback; a post-commit error causes no duplicate retry.	Timeout/unavailable tests for LLM/OCR/STT/Redis, checking the response, pending and DB.
NFR-06	Privacy and security scope	Do not log credentials; delete temporary media; use traits only with consent.	Check logs/config/temp files and consent; note that `default_user` is not yet production multi-user isolated.

Code	Requirement	Target	How to measure
NFR-07	Testability	Pure algorithms independent of DB/LLM; typical and edge cases with hand-computed expected values.	Unit tests for trend, anomaly, runway, correlation, budget, goal, matching and coverage/traceability reports.
NFR-08	Background-task consistency	The same event/job retry creates no duplicate message/export or repeated payment.	Run the same fingerprint multiple times, simulate mid-step failure and compare records; a live Redis worker is measured separately.
NFR-09	Traceability	Insight returns facts, data period, method, provider and degraded components.	Contract tests and reconciliation of FR → formula → test → artifact.
NFR-10	Maintainability and replaceability	The route–service–model modules, provider adapters and configurable thresholds do not change the business contract.	Dependency review, provider/fallback swap tests and checking that no formula is placed in the route/narrator.

The current acceptance limitations include: no production authentication/authorization yet; the grounding checker is still a design target; the Redis worker has not been fully smoke-tested live; the "PDF" flow currently returns HTML; backup/restore lacks sufficient evidence to be declared complete.

3.2. SOFTWARE DESIGN

3.2.1. Application Architecture

PERFIN uses a modular monolith rather than microservices. The Express API contains business modules that are independent at the source level but share the same deployment process. The worker is a separate process for background jobs. This

structure suits the scale of the project, reduces operational cost and still preserves module boundaries.

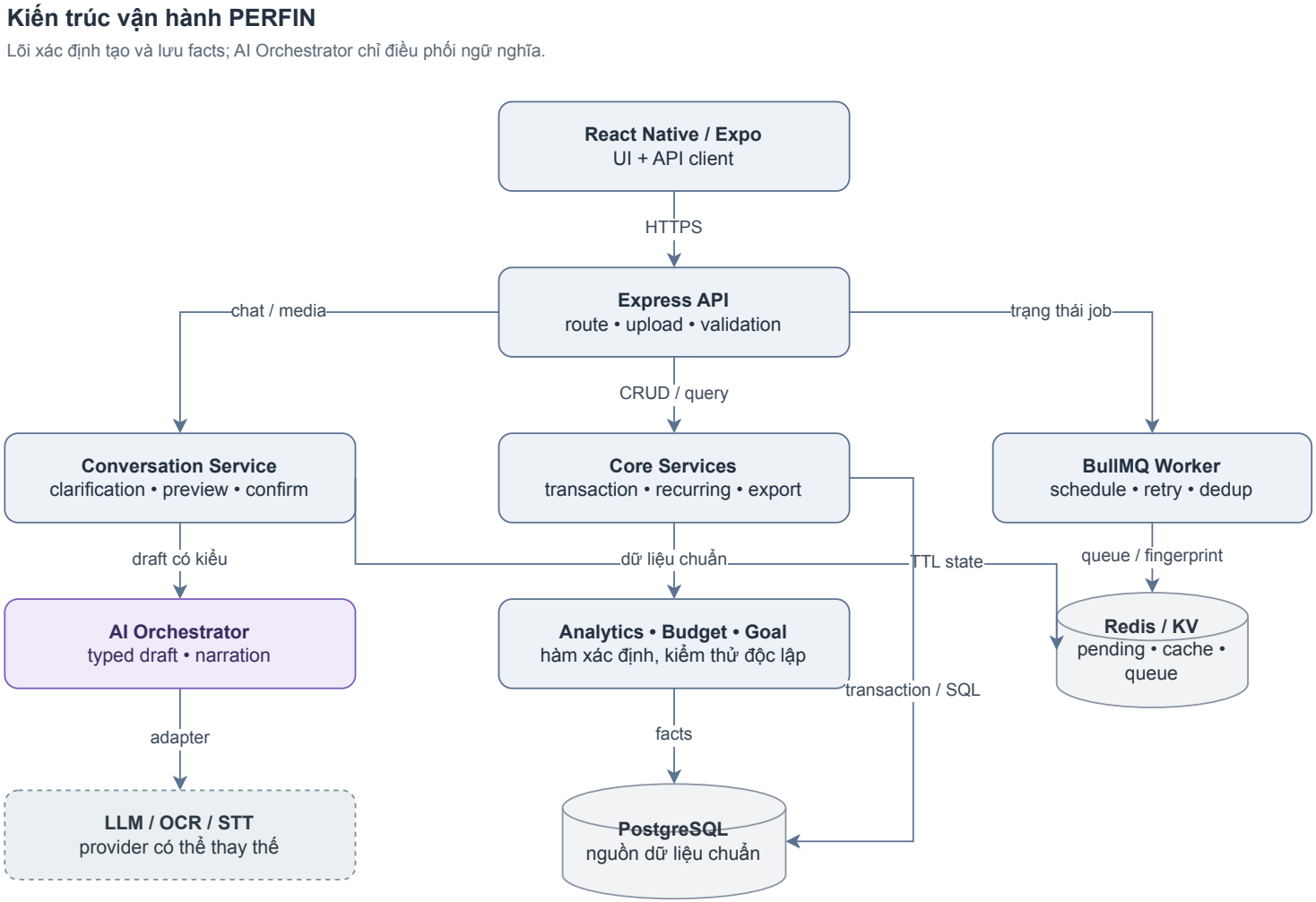

Figure 2: PERFIN's runtime architecture. The deterministic core creates and stores facts; the AI Orchestrator only coordinates semantics.

Figure 2 shows the standard path: mobile calls a route, the route hands off to a service, and the service uses a model to access PostgreSQL. The Analytics Engine, Budget Engine and Goal Planner compute deterministic results. Redis serves cache/state/queue; the worker handles periodic tasks; the AI provider sits outside the data trust boundary.

Table 12: Architecture components and responsibilities

Component	**Main responsibility**	**Not responsible for**
Mobile App	Capture input, preview, display and confirm.	Computing insights or holding provider secrets.
Express Routes	HTTP contract, input validation, response mapping.	Containing analysis formulas.
Core Services	Business rules, transactions and confirmation.	Generating arbitrary answers.
AI Orchestrator	Tool routing, extraction and fallback.	Accessing the DB outside of tools.
Analytics Engine	Computing statistical facts.	Writing personalized advice.
Persona/Narrator	Phrasing facts in a tone.	Changing numbers or deciding.
PostgreSQL Models	Queries, constraints and transactions.	Storing temporary conversation state.
Redis/KV	TTL state, cache and rate count.	Being the authoritative financial data source.
BullMQ Worker	Periodic jobs, retry and dedup.	Handling interactive requests directly.

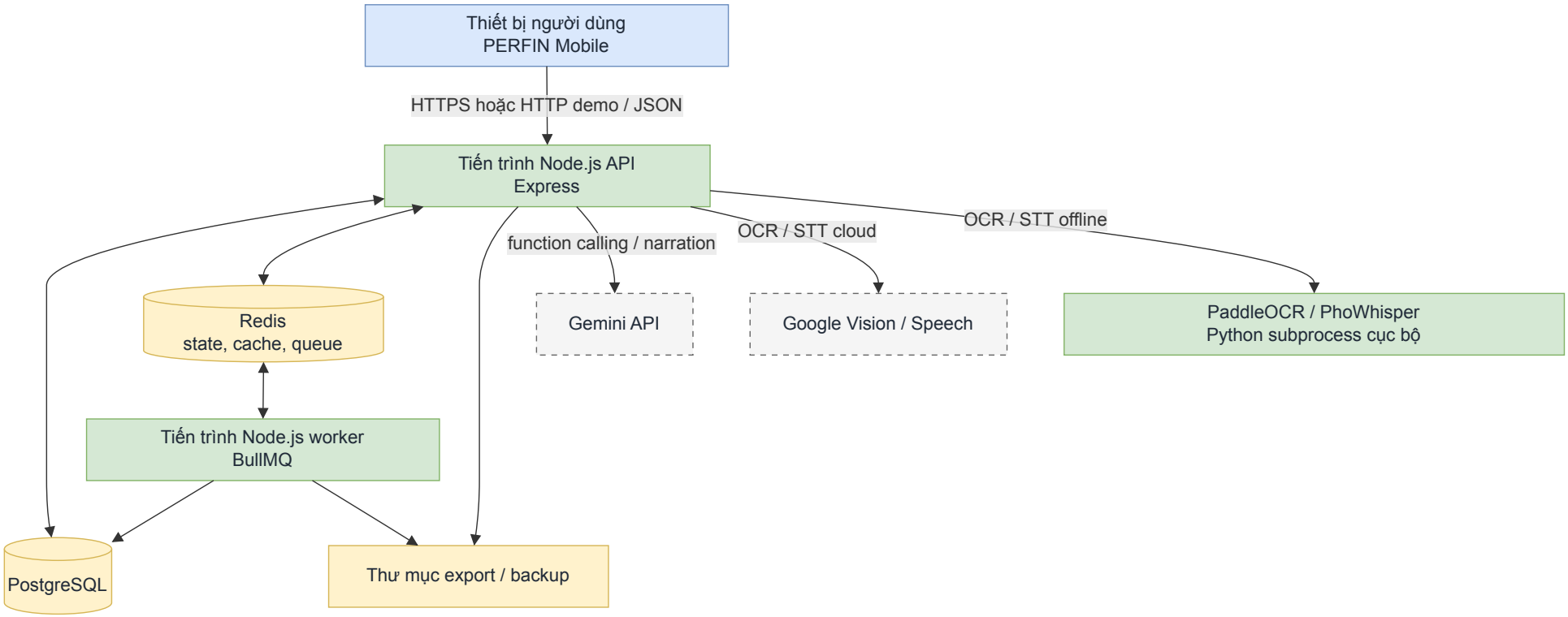

Figure 3: Prototype deployment diagram. The frontend, API, worker, PostgreSQL and Redis belong to the demo environment; the AI provider is called over the network.

Figure 3 is a **prototype deployment design**, not a description of a production cluster or a high-availability commitment. Docker Compose currently only supports Redis; the API, worker and PostgreSQL need to be launched/configured separately. Readiness must be separated from liveness so the backend is not considered ready when DB bootstrap fails.

3.2.2. Data Design

3.2.2.1. Domain model

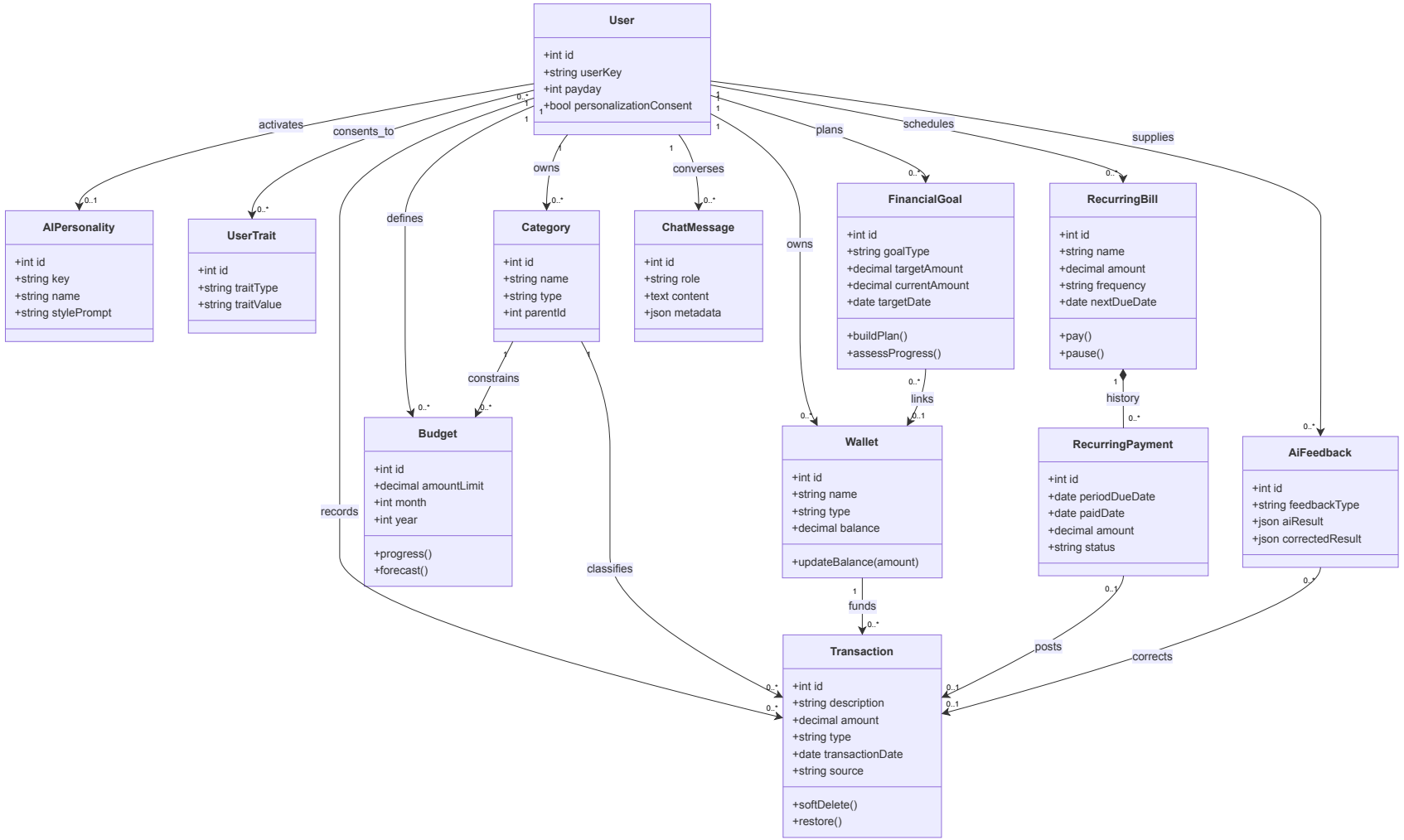

Figure 4: Domain model by business aggregate. The data clusters are separated from the analytics services and the language layer.

Figure 4 separates four aggregates: ledger, planning, recurring bills and conversation–feedback. The User box is repeated as a layout anchor but denotes a single entity; therefore the ownership relations and the fact-reading flow are all straight lines, without crossings. The Analytics/Budget/Goal Services only read aggregates to compute facts or plans and do not own financial entities.

3.2.2.2. Physical model

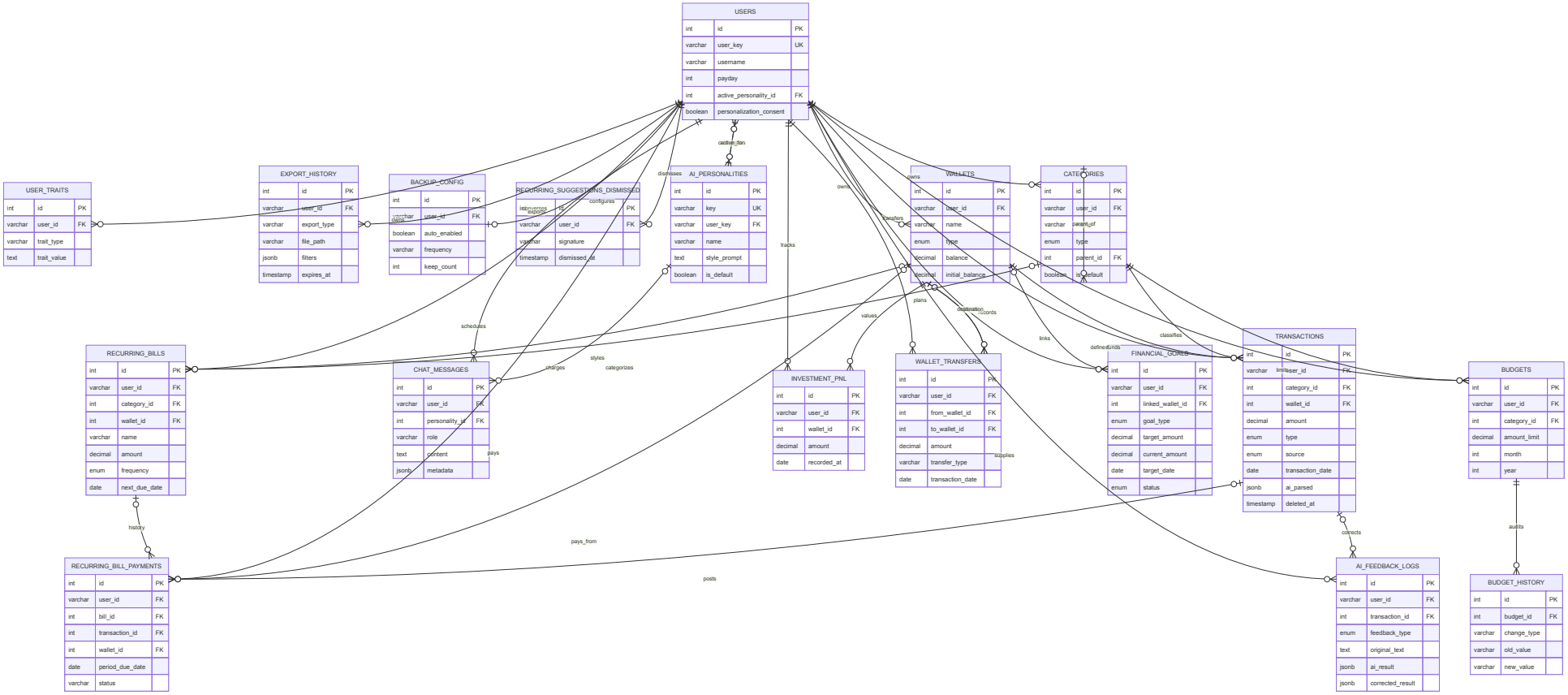

Figure 5: The physical entity–relationship diagram reconciled with the runtime migration chain.

The ERD in Figure 5 has 18 physical tables: `users`, `ai_personalities`, `user_traits`, `categories`, `wallets`, `transactions`, `budgets`, `budget_history`, `chat_messages`, `investment_pnl`, `wallet_transfers`, `export_history`, `backup_config`, three recurring tables, `financial_goals` and `ai_feedback_logs`. Three compatibility views are not counted as tables. A single `users` table is repeated as four reference anchors; cross-module FKs are noted inside the child table box instead of drawing long lines. This presentation keeps PK/FK/UQ/CHECK but eliminates curves and most crossings. The `users.user_key` key is the bridge to `default_user`, not evidence of existing authentication.

Table 13: Main data entity groups

Group	Tables/entities	Meaning
Users and personalization	`users`, `ai_personalities`, `user_traits`	Demo profile, persona, consent and traits.
Personal ledger	`wallets`, `categories`, `transactions`	The core income–expense source.
Budget	`budgets`, `budget_history`	Limits and change history.
Recurring spending	Three recurring tables	Schedule, payments and skipped suggestions.
AI feedback	`ai_feedback_logs`	Initial result and the user's correction.
Goals	`financial_goals`	Saving, purchase, debt and progress.
Special cash flows	`wallet_transfers`, `investment_pnl`	Wallet transfers and investment profit/loss.
Operations	`chat_messages`, `export_history`, `backup_config`	Chat, export and backup configuration.

3.2.2.3. Important data rules

- Money uses `DECIMAL(15,2)`; the service must convert types explicitly when computing in JavaScript.
- Soft delete preserves recoverability; report queries by default exclude deleted records.
- Deleting a category that has transactions must be blocked or use controlled re-tagging, not a cascade that loses history.
- Pending/clarification lives in Redis with its own schema and TTL, not in the

durable ERD.

- Every query by ID in the design target must include `user_id/user_key` to prevent cross-access when authentication is added.

3.2.3. Detailed Design

3.2.3.1. LLM boundary in the architecture

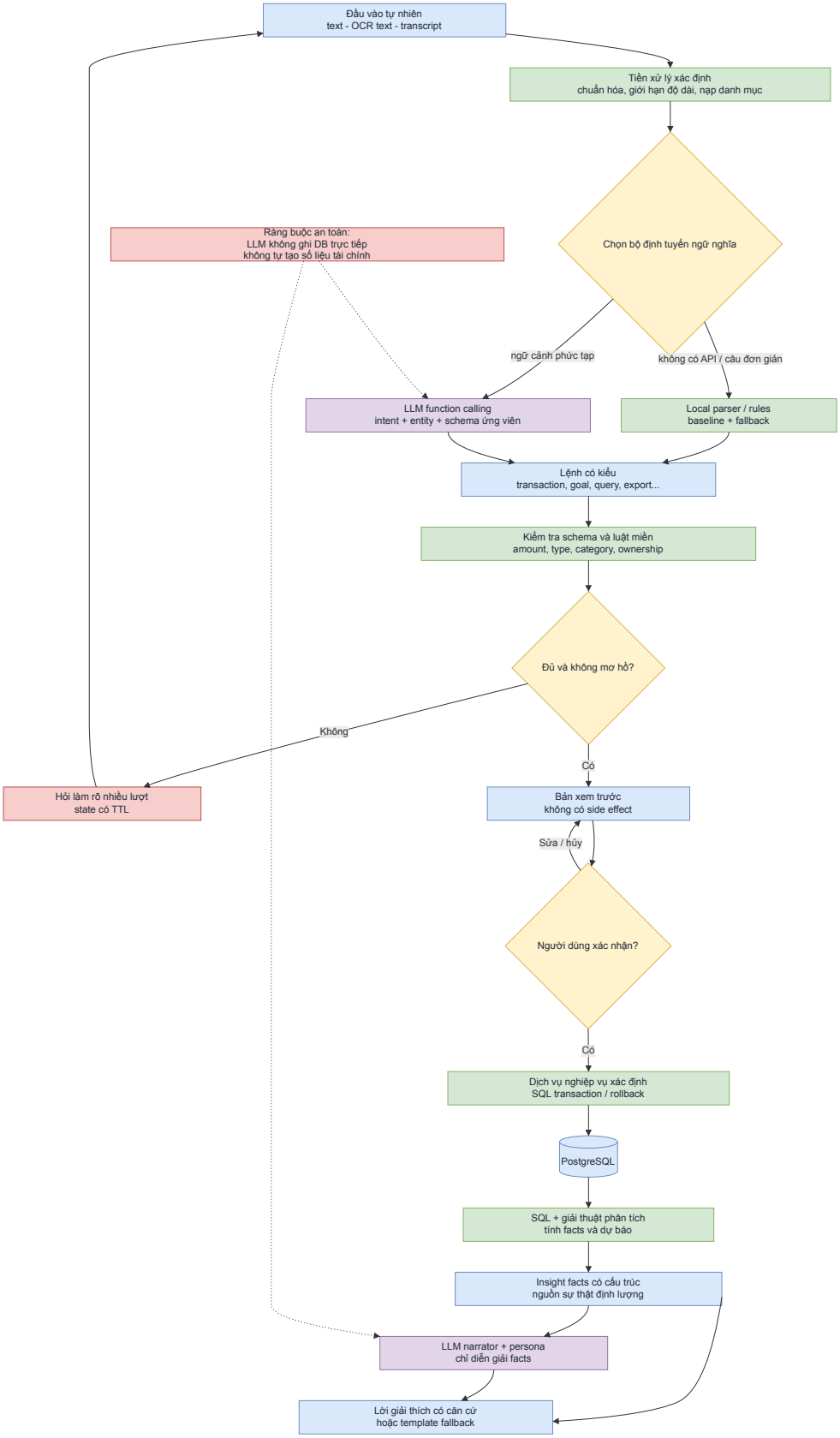

Figure 6: The LLM's responsibility boundary: data and facts are created in the deterministic core; the LLM only performs semantic conversion and interpretation.

Figure 6 is the most important diagram for explaining the LLM's role. Natural input is turned into a typed command; the service validates and creates a preview. For an analytical question, SQL and algorithms create facts before the narrator receives the data. The LLM has no direct connection to PostgreSQL and does not execute tools itself. The LLM's benefit is evaluated by entity accuracy, clarification rate, saved actions and interpretation quality, not by replacing formulas.

3.2.3.2. Conversation state machine

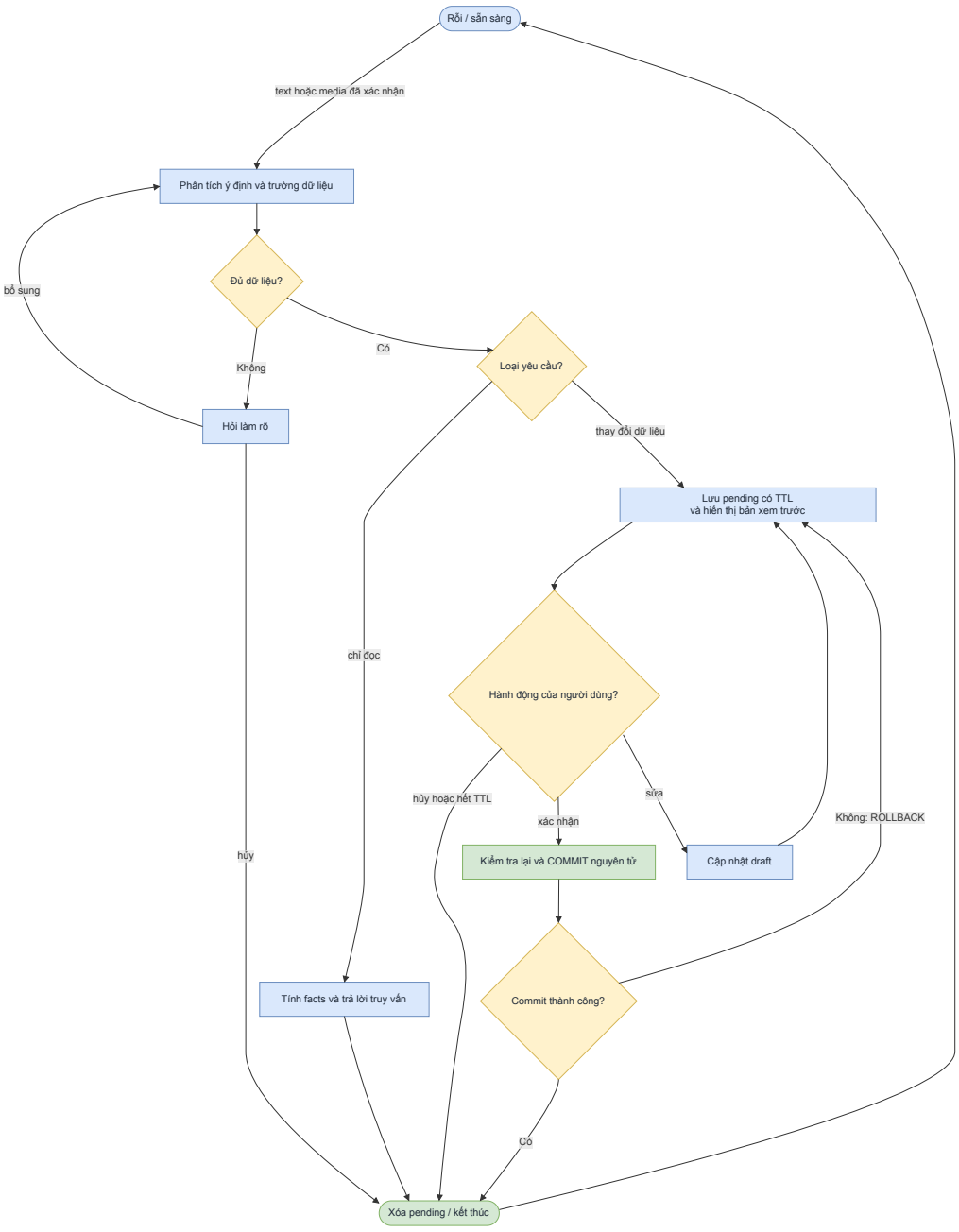

Figure 7: The conversation and pending-transaction state machine.

The main states in Figure 7 are idle, parse, collecting, preview, confirmed, cancelled and expired; candidate choices are held in collecting. Redis keeps the intent, pending fields, collected data and candidates within a limited TTL. The data-writing branch must go through preview then confirmed. The last three states end the current session; a new request creates another idle session. The design target uses an atomic claim operation on confirmation so two simultaneous requests do not both create a transaction.

3.2.3.3. Text transaction entry

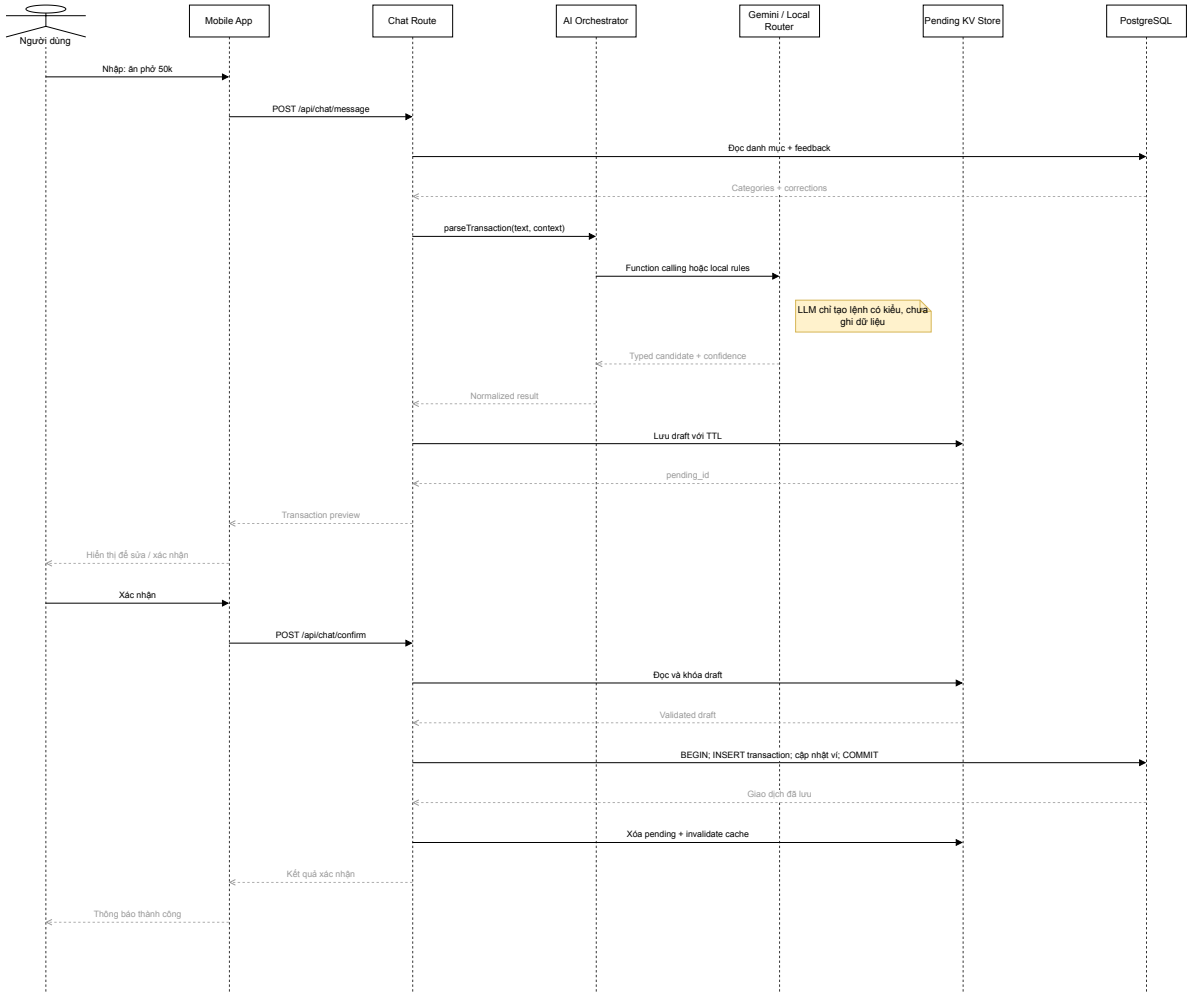

Figure 8: Sequence diagram for text transaction entry. The LLM sits outside the data-writing transaction.

The sequence in Figure 8 consists of receiving text, fetching categories/wallets and corrections, calling the LLM tool or local parser, normalizing, validating, clarifying if incomplete, creating a preview, editing/confirming and writing to the DB in a transaction. The budget check happens after the balance and transaction are consistent. If the commit fails, no success message may be returned.

3.2.3.4. Multimodal input

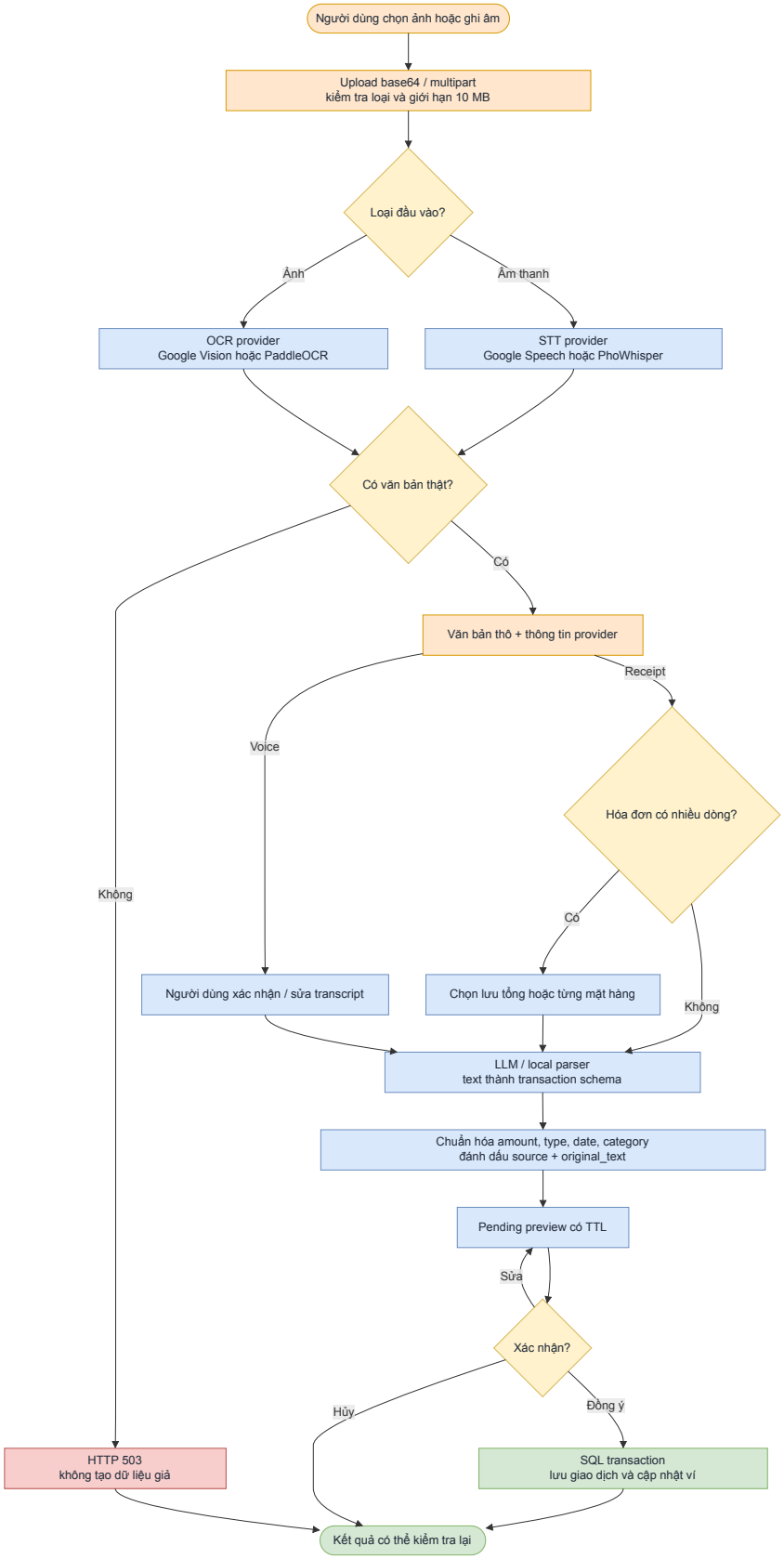

Figure 9: The multimodal input processing flow. Voice and image converge into the shared text pipeline after the confirmation step.

Voice goes through STT and a transcript-confirmation step. Image goes through preprocessing/OCR and then allows choosing the invoice total or individual line items. The two branches converge at the text pipeline. A provider error must be represented as an error or a request for manual entry; no fake raw text is used. The report currently has no media ground truth, so Figure 9 describes the **design target**, not evidence of OCR/STT accuracy.

3.2.3.5. Classification and feedback loop

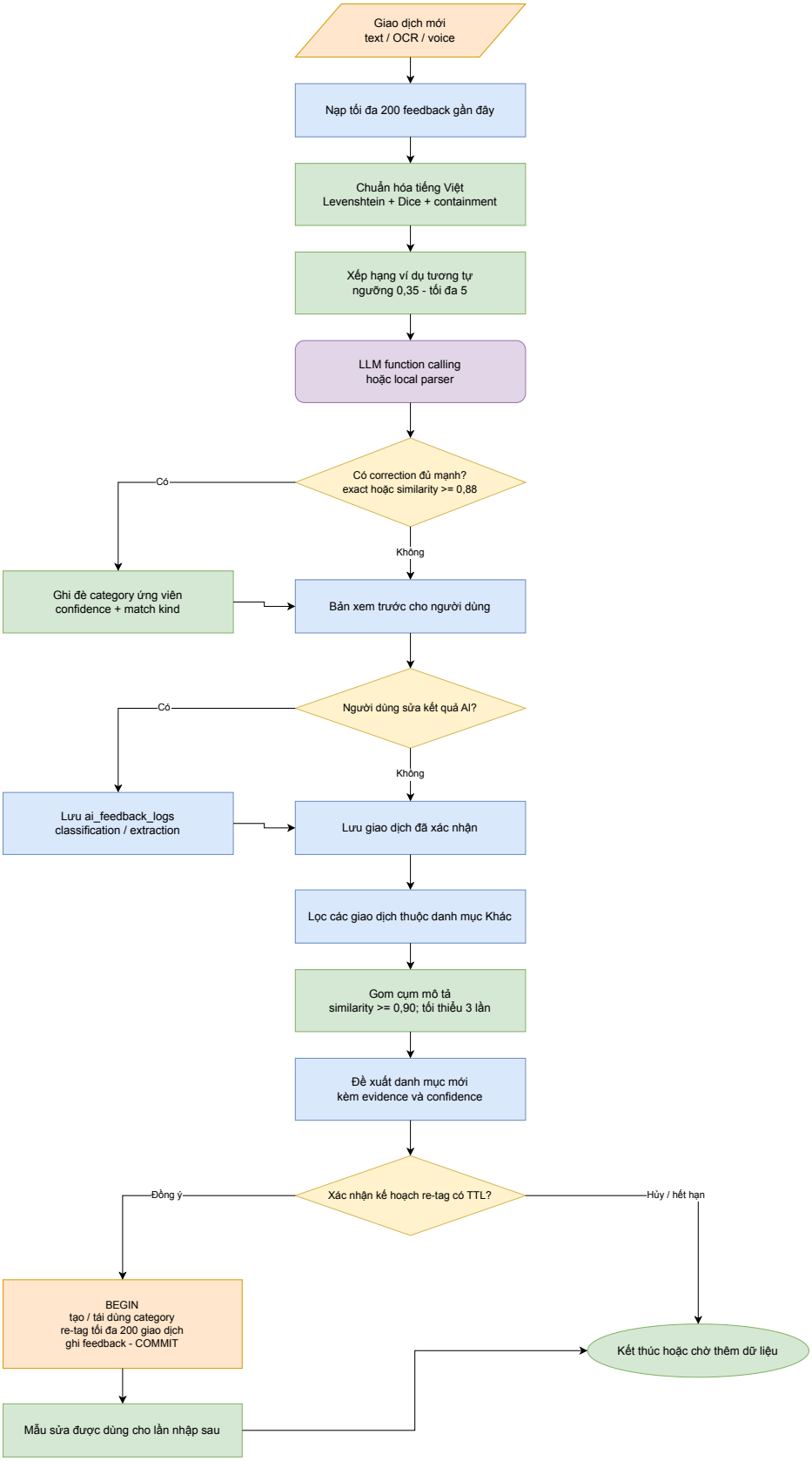

Figure 10: The classification feedback and personalization flow using correction retrieval, not fine-tuning.

As shown in Figure 10, when the user edits a category, the system stores the AI-result–correct-result pair. Next time, the exact/fuzzy correction is considered first, then the matcher/LLM. Repeated transactions in “Other” can be clustered to suggest a category; creating a category and re-tagging is always a plan awaiting confirmation. Conflicting history must lower confidence instead of learning the most recent correction.

3.2.3.6. Grounded insight generation

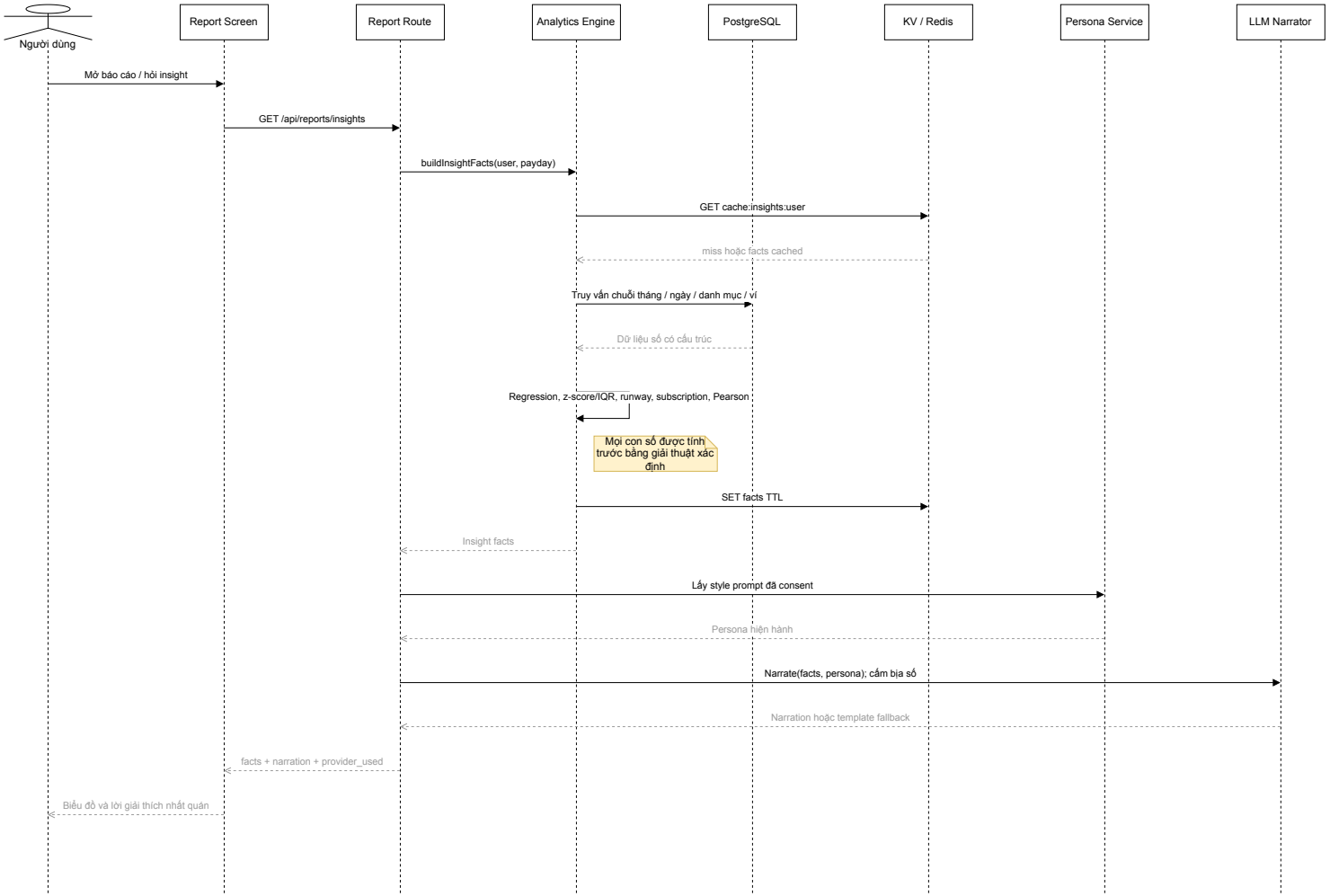

Figure 11: Sequence diagram for grounded insight generation. The response returns both the message and the facts for traceability.

In Figure 11, the SQL model retrieves data; the Analytics Engine computes facts and metadata; a guard checks units and the observation count. Only then does the narrator interpret according to the persona. When the LLM fails, the template narrator uses the same facts. The design target adds a numeric checker before returning the response to detect unsupported numbers.

3.2.3.7. Goal planning

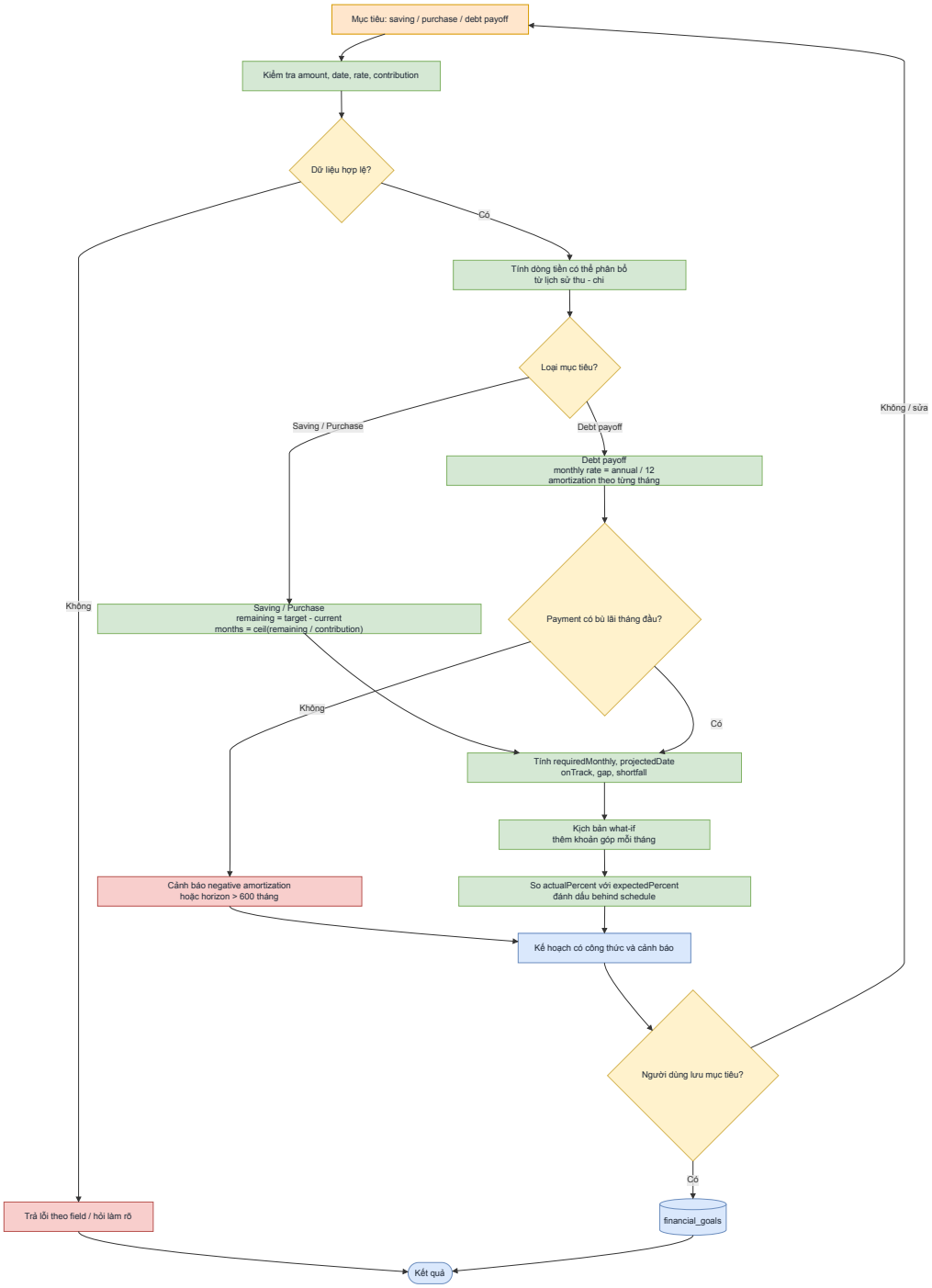

Figure 12: The goal-planning and what-if simulation flow.

The LLM in Figure 12 only extracts the goal, amount, deadline, contribution and interest rate. The Goal Planner checks the data domain, computes surplus, deadline, contribution and warnings. What-if creates a new scenario without changing the original plan. Only a confirmation saves to `financial_goals`.

3.2.3.8. Proactive tasks

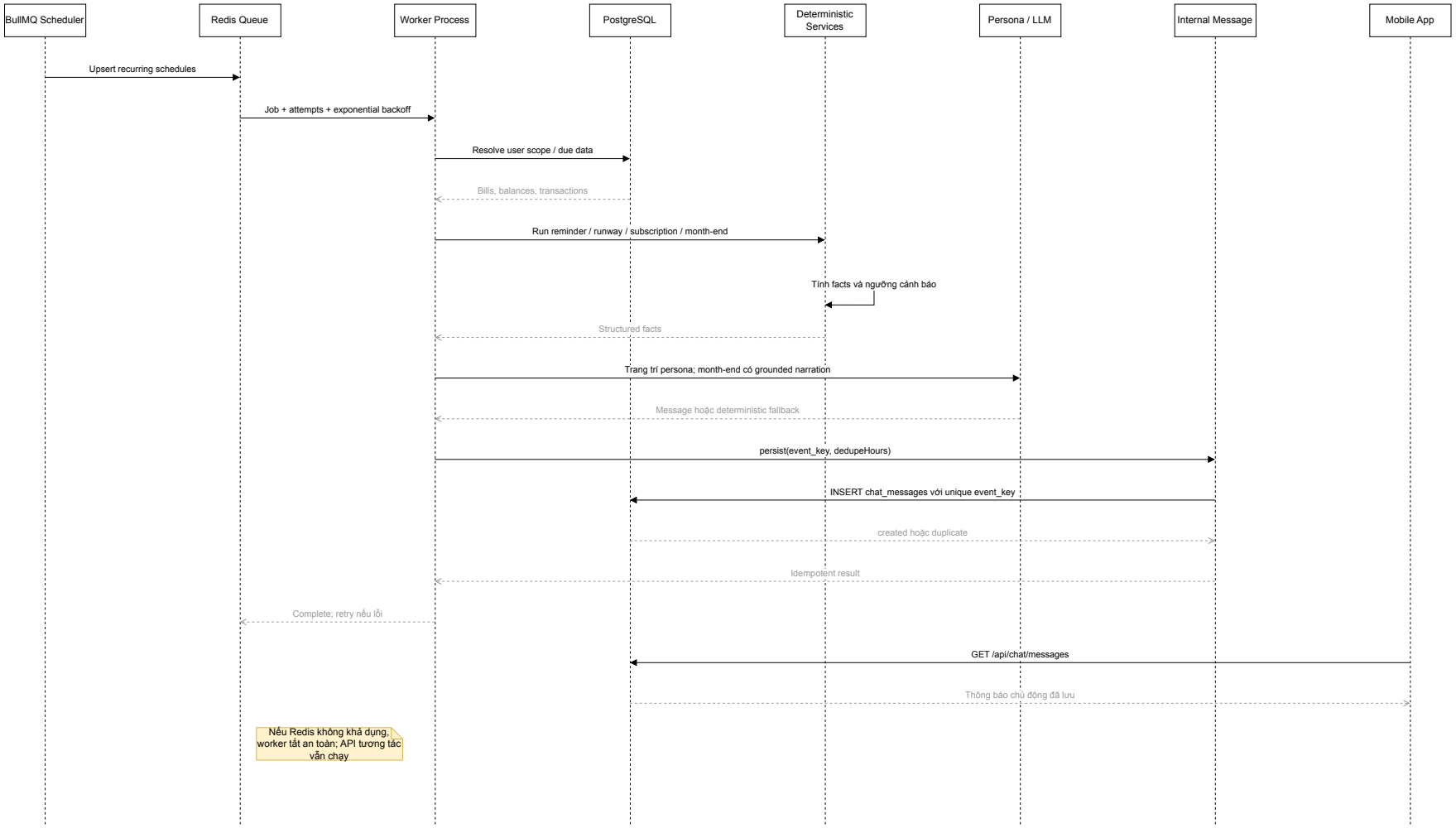

Figure 13: Sequence diagram for proactive tasks and the mechanism preventing duplicate effects on retry.

In Figure 13, the scheduler creates an identified job; the worker takes the correct user scope, calls the handler, computes facts and writes an internal message or export. A retry uses the same fingerprint; a unique event key and idempotent operations eliminate duplicate notifications. The current feature creates a message in `chat_messages`, not a push notification. Scheduled auto-backup has no corresponding worker yet and must not be described as operational.

3.2.3.9. Control of LLM-initiated operations

Table 14: Control conditions for LLM-initiated operations

Operation	Required validation	Confirm	Atomic/idempotent
Create one/many transactions	amount > 0, valid type/category/wallet	Yes	Batch and balance in the same transaction.
Wallet transfer	Two distinct wallets, valid amount/date; negative balance allowed	Yes	Debit, credit, history in the same transaction.
Create recurring bill	Valid name, amount, cycle and date	Yes	No duplicate under the same plan.
Apply budget	Valid history and total limit	Yes	Upsert by user/category/period.
Create goal	Suitable type, target, date, interest	Yes	Save after preview.
Create category and re-tag	Non-generic name, ID belongs to user	Yes	Create/reuse, retag and feedback in the same plan.
Query/insight	Valid data range	No	Traceable facts.
Export	Valid format and date range	Yes from chat	Cleanup with fingerprint.

3.2.4. Security, Privacy and Ethics

The prototype has no production authentication yet; therefore multi-user isolation must not be claimed. The routes use `default_user`. Every ID-based operation has been audited so that it always carries a `user_id` predicate, and cross-access tests now exist (Table 15); but a data predicate is not a substitute for authentication: real authentication and authorization must still be added before a real deployment.

Table 15: Cross-access (IDOR) testing on ID-based endpoints, 25/07/2026

Endpoint under test	Result	Interpretation
`GET/PUT/DELETE /api/budgets/:id`	404	Another user's budget cannot be read, modified or deleted.
`GET/PUT/DELETE /api/recurring/:id`	404	Same for recurring bills.
`POST` on `:id/pause`, `:id/resume`, `:id/pay`	404	No state change or payment can be written onto another user's record.
`GET` on `:id/payments`	404	Before the ownership check was added at the route, this returned 200 with an empty history: no data leak, but it still revealed that the id existed.
Control: the caller's own records	200	Legitimate access is unaffected.

Method: create a second profile, insert records owned by it, then call the API as `default_user` using exactly those IDs; 10/10 cross-access attempts were blocked and 2/2 legitimate accesses still worked. The four regression cases in `tests/user-idor-scope.test.js` assert on the *emitted SQL* rather than only the return value, because with a single demo profile a query missing the `user_id` predicate still returns the expected record. Artifact: `log/idor-verification_2026-07-25.json`.

AI credentials, service accounts and connection strings must not be written to Git or chat logs. Images/audio need a delete-after-processing policy. `user_traits` may be used only when consent is enabled. The persona must not pressure, disparage or turn suggestions into definitive investment advice. Backup must not use an AES key packaged inside the client; the key must come from a secret-management mechanism external to the application.

3.3. TESTING

3.3.1. Test Plan

3.3.1.1. Test Matrix

Table 16: Test matrix

Level	Target	Technique	Evidence
Unit	Analytics, matching, budget, goal, validation	Normal data, boundary, invariant	`node:test`, hand-computed expected values.
Integration	Route–service–model, Redis, PostgreSQL	Test DB, rollback and TTL	Run logs and DB snapshots.
AI contract	Tool schema and parser	Labeled sentence set, provider/fallback	Per-case JSON.
Media	OCR/STT adapter	Real images/audio with ground truth	CER/WER and field accuracy.
Grounding	Narrator/persona	Compare response figures with facts	Numeric faithfulness.
Worker	Retry, deduplication and schedule	Repeat runs with same event, injected errors	Effect counts before–after.
System	Text/voice/image to DB	End-to-end on device	Video/logs and DB assertions.
UAT	Target users	Timed/step-counted tasks	Completion rate, SUS.

3.3.1.2. Test Data Design

The text dataset must separate the development and evaluation sets, covering the units `k`, `nghìn`, `ngàn`, `tr`, `triệu`, numbers in words, multiplication, relative dates, income–expense–transfer–investment, multiple transactions, typos, Vietnamese–English mixing, missing/ambiguous cases and conflicting corrections. The media set must store the original images/audio together with the reference transcript or invoice table. The analytics data should be synthetic/anonymized data with known slope, outliers, cadence and correlation for cross-checking against the formulas.

3.3.1.3. Metric Suite

Table 17: Evaluation metric suite

Group	Metric	Interpretation
Extraction	Precision, recall, F1 per field, exact match	Compare amount/type/date/category/wallet against labels.
Classification	Accuracy, macro-F1, confusion matrix	Do not let high-sample categories mask low-sample ones.
Clarification	Completion rate, number of turns	Count only initially missing/ambiguous cases.
OCR/STT	CER/WER and transaction-field accuracy	Separate raw quality from business impact.
Grounding	Numeric faithfulness, unsupported-number rate	Every figure must map to facts.
Analytics	Error against expected	Test formulas and boundary conditions.
Performance	p50, p95, error rate	Separate cache hit/miss and provider.
Worker	Duplicate-effect rate, retry success	Repeat with the same fingerprint.
Experience	Completion, time, number of operations	Compare chat with form on the same task.

3.3.2. Results and Analysis

3.3.2.1. Measured Results

On 16/07/2026, checks from unit to runtime level were run in the workspace. The results are presented within the scope of their evidence, without extending into conclusions about model accuracy or production readiness.

Table 18: Measured results and missing measurements

Item	Result	Category	Scope of conclusion
Backend `npm test` after fixes	182/182 tests passed, 0 failures	**Measured**	Unit/service across 38 test files; includes transaction, concurrency, recurring, health, importer, time-series and user scope. The 16/07/2026 milestone was 100/100, before the suite was extended.
Local parser quality gate	31/31 strict pass; 0 partial; 0 fail	**Measured**	Local, without Gemini; strict exit code, 31 hard-coded cases.
Baseline before fixes	13/13 test files; parser 27/31 strict	**Measured**	Kept for error analysis and to demonstrate the impact of the fixes.
Full API/DB/media smoke	23/23 checks passed	**Measured**	Live PostgreSQL, preview, edit, cancel, OCR on 2 images and STT on 1 M4A; does not include the Redis worker.

Item	Result	Category	Scope of conclusion
Mobile-web UI smoke	4/4 gates passed at 390×844	**Measured**	Three screens with no overflow; uploaded images render for real; Chromium.
Expo web/Android export	Bundled 653/960 modules	**Measured**	Demonstrates the frontend bundles; not runtime performance.
Financial CSV import	5,265 rows, 0 rejects, full provenance	**Measured**	Run twice on a clone and reconciled against live PostgreSQL after backup.
Category classification (local parser)	Acc 29.36%; macro-F1 0.177 (12 classes)	**Measured**	5,265 labeled rows; historical labels follow the money source, so only partially schema-independent.
Ablation parser vs LLM (Gemini)	Parser 22.2%/0.204; LLM 59.5%/0.607	**Measured**	Stratified sample of 63 sentences; 63/63 Gemini calls succeeded; p50 964 ms.
Feedback before/after	Acc $0\% \rightarrow 68.19\%$; macro-F1 $0 \rightarrow 0.544$	**Measured**	3,502 parser-wrong sentences split seed/holdout; control-group non-regression checked.

Item	Result	Category	Scope of conclusion
OCR/STT accuracy	Not yet published	**Not measured**	Requires images/audio with ground truth.
Redis worker running	Not confirmed in this measurement round	**Not measured**	WSL has no Redis/Docker; API fallback and degraded readiness were checked.
Numeric grounding, p50/p95, UAT	Not yet published	**Not measured**	Requires a checker, environment and version-locked scenarios.

The baseline parser had two amount errors: “1 triệu 5” and the multiplication “3 cái áo, mỗi cái 200k”; two partial cases related to “quần jeans” and “đi ăn”. After the fixes, all four cases have regression tests and the harness reaches 31/31 strict, while also returning a non-zero exit code whenever any case fails. The 100% rate applies only to this fixed set, and is not yet a Gemini benchmark or an independent labeled set.

The full smoke test demonstrates that the HTTP–PostgreSQL flow, Gemini structured output, OCR and STT can complete on the selected fixtures. This result is only a functional smoke test: there is no ground truth sufficient to infer accuracy, CER/WER or numeric faithfulness; a single timing measurement is also not p50/p95. The Redis worker has not been run live in the recorded environment.

3.3.2.2. Status of Critical Features

Table 19: Status and target behavior of critical features

Feature	Current status	Main gap	Completion condition
Transactions and analytics	**Measured**	Transaction, post-commit, zero-fill and full DB smoke passed; 5,265 live rows reconciled.	Labeled dataset and broader DB fault-injection.
Chat preview/confirm	**Measured**	One-time claim, stale ID, edit/cancel race, history and HTTP–DB smoke passed.	Measure TTL/claim with real Redis and higher concurrent load.
Local parser	**Measured** on an independent set	31/31 strict on the fixed gate; on 5,265 independent rows only 29.36% acc / macro-F1 0.177.	Improve low-support categories and ambiguous money-source sentences.
LLM tool routing	**Measured** (ablation)	Gemini 59.5% acc / macro-F1 0.607 vs parser 22.2% / 0.204 on the same 63-sentence sample.	Expand the sample and lock cost/latency per provider.
OCR/STT	**Measured**	Real-provider smoke passed with 2 images and 1 M4A; no ground truth yet.	Adapter failure tests, CER/WER and field accuracy on an anonymized set.
Wallet/transfer	Partially implemented	Missing wallet-creation route/UI on a fresh install; no transfer integration test yet.	Minimal CRUD and balance-integrity testing.

Feature	Current status	Main gap	Completion condition
Recurring and worker	**Measured**	Payment locks rows, is atomic, has an expected period and chat preview; notification is still internal chat.	Live DB tests and an out-of-app reminder channel if expanded.
Export/backup	Misnamed	Denominator/escaping fixed; “PDF” is still HTML, backup still missing.	Real PDF or relabel; safe backup/restore.
Auth/multi-user	Out of current scope	`default_user`; every ID-based operation is now scoped and covered by cross-access tests, but there is no authentication.	Do not demo as present; add authentica-tion/authorization before production.

3.3.2.3. Quantitative Experiments on the Labeled Set

The acceptance run of 16/07/2026 (above) confirmed that the system runs correctly at the runtime level but did not yet quantify algorithmic quality on an independent set; a second measurement round on 24/07/2026 therefore added three quantitative experiments over the full labeled set to answer the specific question of how accurate the algorithm is. These three experiments were run on 24/07/2026 against `dataFinance.csv` itself (5,265 rows, SHA-256 `a9b7cf1b...027ca94f`), commit `5f03476`, Node v24.16.0, provider `gemini`, model `gemini-3.1-flash-lite`. Code and reproducible artifacts (JSON + Markdown) live under `demo/backend/tests/experiments/` and `log/`.

a) Category-classification benchmark — local parser on the full set. Running `parseLocalTransaction` over all 5,265 labeled rows and comparing the predicted label with the gold label mapped from the historical taxonomy.

Table 20: Local-parser classification results on `dataFinance.csv`

Metric	Value
Accuracy (micro)	29.36%
Macro-F1 (12 classes with support)	0.177
Weighted-F1	0.301
Accuracy (excluding the "Khác" class)	27.52%
Misclassified cases	3,719 / 5,265
Misses involving the "Khác" class	2,957

The low figure is a **meaningful finding, not a failure**: historical labels classify by *money source* (e.g. "mẹ cho tiền ăn" → the family label "Khác") whereas the parser classifies by *content* (→ "Ăn uống"). The two schemes use different axes, so 79.5% (2,957/3,719) of misses revolve around the "Khác" class. This is precisely why the design chooses the LLM as the primary extraction layer and keeps the user in a confirmation loop rather than trusting alias matching.

b) Local parser vs LLM (Gemini) ablation. On a stratified sample of 63 sentences (5 per class, seed 42), comparing the two category-extraction arms; the LLM arm calls `parseWithGemini` directly, bypassing the cache and local routing.

Table 21: Local parser vs LLM ablation

Arm	Acc	Macro-F1	Clarify	p50 (ms)	API calls
Local parser	22.2%	0.204	84.1%	0	0
LLM (Gemini)	59.5%	0.607	95.2%	964	63

The LLM generalizes roughly 3.0× better on macro-F1 over free-form sentences, at the cost of a p50 latency of 964 ms and per-call API cost. Both arms pass through the confirmation step before any write, so the LLM is an extraction-assist layer rather than a source of truth. All 63 Gemini API calls succeeded (0 errors); of these, 42 sentences received a definite label and 21 returned empty and are counted as misclassifications. The accuracy above is computed over the full 63-sentence sample.

c) Feedback before/after (correction retrieval). Excluding the "Khác" class leaves 4,832 content sentences. The parser-wrong group (3,502 sentences) is split in half:

seed (write corrections) and holdout (replay). Improvement is measured on the holdout, with a no-regression check on the control group (1,330 sentences the parser already got right).

Table 22: Feedback before/after on the holdout set

Metric	**Before (parser)**	**After (correction → parser)**	Δ
Accuracy	0.00%	68.19%	+68.19 pts
Macro-F1	0.0000	0.5440	+0.5440
Weighted-F1	0.0000	0.7824	+0.7824

1,194 sentences flipped wrong→right through text-similarity retrieval (not exact match, so it measures generalization to near-identical sentences). The control group had 466/1,330 sentences turned wrong by corrections, but 430 of those (92%) are **inherent label noise** in the historical data — the same text appears with several different gold labels, which no system can disambiguate. Genuine regression on unambiguous sentences is only 36 cases (2.7%), confirming the corrections do not systematically mislearn.

d) Grounded narration. Not measured quantitatively in this round; kept at design level (give the same facts to multiple personas, check that figures stay unchanged as the tone varies) and is a priority extension.

Results (a)–(c) have reproducible logs and are therefore reflected in the abstract and conclusion; (d) is not yet eligible.

CHAPTER 4: CONCLUSION

4.1. RESULTS ACHIEVED

At the design and prototype-implementation level, PERFIN has established an architecture that separates data, algorithms, and the LLM layer; it includes migrations for 18 physical tables; it has analytics, feedback, budget, goal, state, and worker modules; and it has a runnable automated test suite. The most important contribution of this report is clarifying the role of the LLM: the model does not replace SQL or statistical algorithms, but instead converts natural language into typed commands and converts facts into explanations.

Table 23: Comparison of Objectives Against Evidence

Objective	Available Evidence	Cautious Conclusion
O1 — Data model	Migrations, models, validation, and the transaction flow.	Implemented at prototype level; fault injection on a real database is still needed.
O2 — Multimodal pipeline	Tool schema, parser, media adapters, preview/state, plus an ablation on a labeled set.	Flow implemented and now measured: on a stratified 63-sentence sample, local parser macro-F1 0.204 and Gemini 0.607; OCR/STT accuracy not yet version-locked.
O3 — Analytics algorithms	Analytics, goal, budget, and feedback functions plus automated tests; feedback measured before/after.	182/182 tests pass; correction retrieval lifts holdout accuracy 0% $\rightarrow$ 68.19%. Broader labeled data still needed.
O4 — LLM boundary	Tool declarations, the facts–narrator flow, and fallback.	The design is clear; numeric faithfulness has not yet been measured systematically.
O5 — Operational evaluation	Regression, parser, full API/DB/media smoke tests, mobile UI smoke tests, and Expo export have all run.	The demo core has runtime evidence; conclusions about the Redis worker, accuracy, performance, or UAT remain insufficient.

The baseline recorded before stabilization showed 13/13 backend files passing and the local parser reaching 27/31 strict cases. After stabilization, the backend reached 182/182 tests, the local-parser quality gate reached 31/31 strict cases, the full smoke suite reached 23/23, and the Expo web/Android export processed 653/960 modules. The demo PostgreSQL database contains 5,265 reconciled transactions, and the mobile-web smoke test confirmed that images display directly in the chat. Three quantitative experiments on this dataset (Section 3.3.2) answer the question of how accurate the algorithm is: the local parser reaches macro-F1 0.177 over all 5,265 rows; in the ablation on a stratified 63-sentence sample, the LLM reaches 0.607 against the parser's 0.204 on the *same sample* (about 3.0×); and correction retrieval raises holdout accuracy to 68.19% — quantifying the core contribution and justifying the human-in-the-loop, LLM-primary design. These figures should not be used to claim that the system is production-ready.

4.2. IMPACT OF THE LLM

The LLM creates value in three respects. First, it reduces input friction by understanding varied phrasing, relative dates, and sentences with multiple intents. Second, function calling turns natural-language sentences into typed commands, allowing forms and chat to share the same business services. Third, grounded narration interprets facts according to context and persona, making algorithmic results more accessible.

This impact is only positive when four safeguards are in place: schema, validation, a preview/confirmation step, and facts grounding. Without these safeguards, the LLM increases the risk of incorrect data; if all computation is delegated to the model, the results lose reproducibility. Therefore, the benefit of the LLM must be measured against parsers and forms using entity F1, completion rate, number of operations, latency, cost, and the unsupported-number rate.

4.3. LIMITATIONS

1. The runtime uses `default_user`; production-grade authentication and authorization are not yet in place.
2. Extraction quality depends on the provider, colloquial phrasing, and image/audio quality; no labeled benchmark exists yet.
3. The anomaly, fuzzy-matching, recurring, and correlation thresholds are initial configurations that have not been optimized on representative data.
4. The runway and trend calculations zero-fill missing days/months but remain simple extrapolations that do not model seasonality, future events, or irregular income.

5. The persona may make responses easier to read, but its behavioral effectiveness has not been demonstrated through UAT.
6. Several supporting demo features remain incomplete: wallet creation, genuine PDF generation, periodic backup, restore, and push notifications.
7. The API–PostgreSQL flow and media providers have been smoke tested; the Redis worker, live retry/idempotency, and multiple mobile devices have not yet been verified.
8. Every ID-based operation has now been audited and given a `user_id` predicate (see Section 3.3), but the system still runs on a single `default_user` profile: that predicate is an upgrade path towards multi-user support, not an authenticated authorization mechanism.

4.4. FUTURE DIRECTIONS

The development sequence must keep the focus on data and algorithms:

1. **Verify core integration:** transactions, pending claims, recurring periods, the live PostgreSQL database, and per-ID user scope already have regression/smoke coverage; the next steps are the Redis worker and fault injection.
2. **Expand the language baseline:** the 31-sentence quality gate has been achieved; an independent evaluation set, typos, multi-intent sentences, and macro-F1 reporting still need to be added.
3. **Verify algorithms:** the time axis, zero-spend days, and denominators have been fixed; a larger synthetic dataset, backtesting, and hand-computed expected values are still needed.
4. **Measure LLM impact:** build a labeled Vietnamese dataset, run a parser–LLM ablation study, and implement a numeric grounding checker.
5. **Verify media handling:** collect anonymized images/audio, measure CER/WER and transaction-field accuracy, and test provider failure cases.
6. **Complete supporting features:** wallet creation, properly formatted PDFs, safe backup/restore, and periodic workers; to be done only after the core is stable.
7. **Prepare for real deployment:** authentication, authorization, audit logging, key management, and media retention/deletion policies.

Bank synchronization, shared wallets, and high availability should only be considered at a later stage. Expanding features before the data correctness and experimental evidence are stable would shift the focus away from the purpose of a basic-topic undergraduate project.

4.5. OVERALL CONCLUSION

PERFIN is academically meaningful when presented as a financial data-processing system with an LLM communication layer, not as a chatbot that happens to do accounting. The appropriate structure is: relational data as the source of truth, deterministic algorithms that produce facts, the LLM assisting with understanding and interpretation, and the user retaining the authority to confirm every change. This report simultaneously indicates what has been implemented, what has been measured, and what gaps remain to be addressed; this is the basis for continued development without compromising the integrity of the results.

BIBLIOGRAPHY

[1] A. Lusardi and O. S. Mitchell, "The Economic Importance of Financial Literacy: Theory and Evidence," *Journal of Economic Literature*, vol. 52, no. 1, pp. 5–44, 2014.

[2] A. Vaswani *et al.*, "Attention Is All You Need," in *Advances in Neural Information Processing Systems*, vol. 30, pp. 5998–6008, 2017. [Online]. Available: https://arxiv.org/abs/1706.03762

[3] Gemini Team, "Gemini: A Family of Highly Capable Multimodal Models," *arXiv preprint arXiv:2312.11805*, 2023.

[4] Google AI for Developers, "Function calling with the Gemini API," 2026. [Online]. Available: https://ai.google.dev/gemini-api/docs/function-calling. [Accessed: 15-Jul-2026].

[5] R. Smith, "An Overview of the Tesseract OCR Engine," in *Proc. 9th International Conference on Document Analysis and Recognition*, vol. 2, pp. 629–633, 2007.

[6] A. Radford *et al.*, "Robust Speech Recognition via Large-Scale Weak Supervision," *arXiv preprint arXiv:2212.04356*, 2022. [Online]. Available: https://arxiv.org/abs/2212.04356

[7] PostgreSQL Global Development Group, "PostgreSQL Documentation — Transactions and Data Definition," 2026. [Online]. Available: https://www.postgresql.org/docs/current/tutorial-transactions.html and https://www.postgresql.org/docs/current/ddl.html. [Accessed: 15-Jul-2026].

[8] Redis Ltd., "EXPIRE," *Redis Commands Documentation*, 2026. [Online]. Available: https://redis.io/docs/latest/commands/expire/. [Accessed: 15-Jul-2026].

[9] Taskforce.sh, "BullMQ Documentation: Idempotent Jobs and Retrying Failing Jobs," 2026. [Online]. Available: https://docs.bullmq.io/patterns/idempotent-jobs and https://docs.bullmq.io/guide/retrying-failing-jobs. [Accessed: 15-Jul-2026].

[10] J. W. Tukey, *Exploratory Data Analysis*. Reading, MA, USA: Addison-Wesley, 1977.

[11] IEEE Computer Society, *IEEE Recommended Practice for Software Requirements Specifications*, IEEE Std 830-1998, 1998.

[12] R. H. Thaler and C. R. Sunstein, *Nudge: Improving Decisions About Health, Wealth, and Happiness*. New York, NY, USA: Penguin Books, 2009.

[13] V. I. Levenshtein, "Binary Codes Capable of Correcting Deletions, Insertions, and Reversals," *Soviet Physics Doklady*, vol. 10, no. 8, pp. 707–710, 1966.

[14] L. R. Dice, "Measures of the Amount of Ecologic Association Between Species," *Ecology*, vol. 26, no. 3, pp. 297–302, 1945.

[15] D. C. Montgomery, E. A. Peck, and G. G. Vining, *Introduction to Linear Regression Analysis*, 5th ed. Hoboken, NJ, USA: Wiley, 2012.

[16] K. Pearson, "Note on Regression and Inheritance in the Case of Two Parents," *Proceedings of the Royal Society of London*, vol. 58, pp. 240–242, 1895.

[17] Money Lover, "Money Lover — Money Manager, Budget Expense Tracker," 2026. [Online]. Available: https://moneylover.me. [Accessed: 24-Jul-2026].

[18] MISA JSC, "MISA MoneyKeeper — a personal expense-management app," 2026. [Online]. Available: https://www.misa.vn/moneykeeper. [Accessed: 24-Jul-2026].

CHAPTER A: LIST OF DRAW.IO DIAGRAMS

Each diagram has its canonical editable source at `archive/latex/figures/drawio` and its rendered version at `archive/latex/figures/rendered`. The report links to the vector PDF version; PNG/SVG formats are used for quick viewing and web documentation. Terminology in the diagrams must be consistent with the report and must clearly distinguish the deterministic core, the LLM, persistent data, transient state, and external services.

No.	Figure Name	Basename	Location in Report
1	System context	`01-system-context`	Figure 1
2	Runtime architecture	`02-runtime-architecture`	Figure 2
3	Prototype deployment	`03-deployment`	Figure 3
4	Business domain aggregate	`04-domain-class`	Figure 4
5	Physical ERD	`05-physical-erd`	Figure 5
6	LLM boundary	`06-llm-boundary`	Figure 6
7	Conversation state	`07-conversation-state`	Figure 7
8	Text-based transaction entry	`08-text-sequence`	Figure 8
9	Multimodal input	`09-multimodal-flow`	Figure 9
10	Feedback and classification	`10-feedback-flow`	Figure 10
11	Evidence-based insight	`11-insight-sequence`	Figure 11
12	Goal planner and what-if	`12-goal-flow`	Figure 12
13	Proactive task	`13-worker-sequence`	Figure 13

CHAPTER B: TEST REPRODUCTION COMMANDS

The commands below must be run from the correct component directory. The person carrying out the test must record the commit, runtime version, provider configuration, and service status; secrets must not be copied into the record.

```
cd demo/backend
npm test
npm run test:ai

cd ../frontend
EXPO_PUBLIC_API_URL=http://127.0.0.1:3000 \
  npx expo export --platform web \
  --output-dir /tmp/perfin-web-final
```

Integration testing requires a dedicated test PostgreSQL instance and a running Redis instance. Migrations must be run against the test database; after each test group, rollback must be verified and data cleaned up using fixtures. A personal development database must never be used as the automated test environment.